

# Kuantum Algoritmalarında Veri Kodlama ve Kuantizasyon Arasındaki İlişkinin Analizi ve Keçeci Layout ile Max-Cut Problemi

Mehmet Keçeci

[mkececi@yaani.com](mailto:mkececi@yaani.com)

ORCID:  <https://orcid.org/0000-0001-9937-9839>, Independent Researcher

Received: 06.08.2025

**Abstract:** Kuantum hesaplama ve özellikle kuantum makine öğrenmesi (KML, QML) alanları, karmaşık problemleri çözme potansiyeliyle büyük bir ilgi görmektedir. Bu potansiyeli hayata geçirmenin önündeki temel zorluklardan biri, klasik verilerin kuantum durumlarına verimli ve anlamlı bir şekilde kodlanmasıdır. Bu süreç, "veri kodlama" veya "gömme" (embedding) olarak bilinir ve kuantum algoritmasının genel performansı üzerinde belirleyici bir rol oynar. Buna paralel olarak, özellikle Gürültülü Orta Ölçekli Kuantum (NISQ) çağında, donanım sınırlamaları ve gürültüye karşı dayanıklılık ihtiyacı, veri temsillerinin optimize edilmesini zorunlu kılmaktadır. Bu noktada, klasik hesaplama aşına olunan "kuantizasyon" (nicemleme), yani sürekli veya yüksek hassasiyetli verilerin daha az bit ile temsil edilen ayrık değerlere dönüştürülmesi kavramı, kuantum bağlamında yeni bir önem kazanmaktadır. Bu çalışma, kuantum algoritmalarında veri kodlama stratejileri ile veri kuantizasyonu arasındaki simbiyotik ve karmaşık ilişkiyi kapsamlı bir şekilde analiz etmektedir. Araştırmanın temel tezi, bu iki sürecin birbirinden bağımsız olarak ele alınamayacağı ve performans, kaynak verimliliği ve gürültü toleransı arasında hassas bir denge kurmak için bütüncül bir yaklaşımla tasarlanmaları gerektiğidir. Analiz, temel kodlama (basis encoding), genlik kodlama (amplitude encoding) ve açı kodlama (angle encoding) gibi yaygın kullanılan veri kodlama yöntemlerini merkeze almaktadır. Her bir kodlama tekniğinin, kuantizasyonun neden olduğu bilgi kaybına karşı ne kadar hassas olduğu ve bu etkileşimin kübit sayısı, devre derinliği gibi kaynak gereksinimlerini nasıl değiştirdiği incelenmektedir. Örneğin, genlik kodlama logaritmik kaynak verimliliği sunarken, veri noktalarındaki küçük değişikliklere karşı oldukça duyarlı olabilir; bu da agresif kuantizasyon stratejilerini riskli hâle getirebilir. Öte yandan, açı kodlama daha sağlam bir yapı sunsa da genellikle daha fazla kuantum kapısı gerektirir. Çalışma, veri kuantizasyonunun yalnızca bir veri sıkıştırma tekniği olarak değil, aynı zamanda belirli kodlama yöntemleri altında bir tür düzenleştirme (regularization) mekanizması olarak da işlev görebileceğini öne sürmektedir. Kuantizasyon seviyesinin ayarlanmasıyla, bir KML modelinin genelleme yeteneğinin artırılacağı ve gürültülü kuantum donanımlarında daha istikrarlı sonuçlar elde edilebileceği tartışılmaktadır. Sonuç olarak, bu analiz, kuantizasyon ve veri kodlama arasındaki etkileşimin anlaşılmasının, pratik ve verimli kuantum algoritmaları geliştirmek için kritik olduğunu ortaya koymaktadır. Bu ilişkiyi optimize etmek, mevcut ve yakın gelecekteki kuantum işlemcilerinin potansiyelini en üst düzeye çıkarmak için temel bir adımdır. Çalışma, araştırmacılara ve geliştiricilere, kendi problem alanlarına ve donanım kısıtlarına en uygun hibrit kodlama-kuantizasyon stratejilerini tasarlamaları için teorik bir çerçeve ve pratik bir rehber sunmayı amaçlamaktadır.

**Keywords:** Kuantum Hesaplama, Kuantum Makine Öğrenmesi, KML, QML, Kuantizasyon, Nicemleme, Kübit, Yapay Zekâ, Gürültülü Orta Ölçekli Kuantum, NISQ, Kuantum Algoritmaları, Kuantum Dolanıklığı, Nicemleme Hatası, Keçeci Layout, Max-Cut Problemi, Keçeci Sayıları, Keçeci Varsayımı.

## I. Kuantizasyonun Kavramsal Tekâmülü: Planck'tan Derin Öğrenmeye Uzanan Bir Yolculuk

Kuantizasyon, en temel anlamıyla, sürekli bir değerler kümesini daha küçük ve ayrık bir değerler kümesiyle temsil etme işlemidir. Modern bilişim ve mühendislik dünyasında her yerde karşımıza çıkan bu kavram, dijital ses ve görüntü işlemeden veri sıkıştırmaya, verimli yapay zekâ modellerinden kuantum hesaplama [26–43] kadar geniş bir yelpazede kritik bir rol oynamaktadır. Ancak bu pratik uygulamaların ardında, 20. yüzyılın başlarında fizikte devrim oluşturan temel bir fikrin yeniden yorumlanarak farklı disiplinlere yayılan büyüleyici bir gelişim yatmaktadır. Bu yolculuk, Max Planck'ın bir "çaresizlik eylemi (act of desperation)" olarak tanımladığı kuantum hipoteziyle başlamış ve günümüzde en karmaşık derin öğrenme modellerini mobil cihazlarda çalıştırabilen bir optimizasyon tekniğine dönüşmüştür. Kuantizasyonun bu disiplinler arası gelişimi, temel bir bilimsel konseptin, teknolojinin ve teorinin ihtiyaçları doğrultusunda nasıl yeniden doğup şekillenebildiğinin çarpıcı bir örneğidir.

### Fiziksel Bir Zorunluluk Olarak Doğuş: Planck ve Kuantum Hipotezi

Kuantizasyon kavramının kökeni, 19. yüzyılın sonlarında fizikçileri meşgul eden "kara cisim ışıması (black-body radiation)" problemine dayanır. Klasik fizik yasaları, ısıtılan bir cismin yaydığı elektromanyetik ışımanın spektrumunu, özellikle yüksek frekanslarda (morötesi bölge) doğru bir şekilde açıklayamıyordu ve bu durum "morötesi felaket ("ultraviolet catastrophe")" olarak biliniyordu (Kuhn, 1978) [1]. 1900 yılında Alman fizikçi Max Planck, bu problemi çözmek için radikal bir öneri sundu. Planck, enerjinin sürekli bir akış halinde değil, "kuanta (quanta)" adını verdiği ayrık paketler halinde soğurulup yayılabileceğini varsaydı (Planck, 1901) [2]. Bu hipoteze göre, bir osilatörün enerjisi, temel bir enerji birimi olan  $h\nu$ 'nin (burada  $h$  Planck sabiti,  $\nu$  ise frekanstır) tam katları şeklinde olabilirdi.

Planck'ın başlangıçta yalnızca matematiksel bir hile olarak gördüğü bu fikir, modern fiziğin temel taşı hâline geldi. Albert Einstein, 1905'te fotoelektrik etkiyi açıklamak için Planck'ın kuantum fikrini kullanarak ışığın kendisinin "foton" adı verilen enerji paketçiklerinden oluştuğunu öne sürdü. Niels Bohr ise bu fikri atom modeline uygulayarak elektronların çekirdek etrafında yalnızca belirli, yani kuantize olmuş enerji seviyelerine sâhip yörüngelerde bulunabileceğini gösterdi. Böylece kuantizasyon, doğanın temel bir özelliği olarak kabul edildi: mikroskobik dünyada enerji, momentum ve diğer fiziksel nicelikler sürekli değil, ayrık değerlere sahipti. Bu, fiziksel gerçekliğin temelden kuantize olduğu, yâni sayısallaştırılmış olduğu anlamına geliyordu.

## Mühendislikte Bir Devrim: Sinyal İşleme ve Darbe Kod Modülasyonu

Fiziğin temel bir prensibi olarak ortaya çıkan kuantizasyon fikri, on yıllar sonra tamamen farklı bir alanda, mühendislikte yeniden hayat buldu. 20. yüzyılın ortalarında telekomünikasyon ve elektroniğin yükselişiyle birlikte, analog sinyallerin (ses, görüntü vb.) dijital olarak temsil edilmesi ihtiyacı doğdu. Analog sinyaller doğaları gereği süreklidir; yâni sonsuz sayıda ara değer alabilirler. Bunları dijital bir sistemde (bilgisayar gibi) saklamak ve işlemek için sonlu sayıda değere dönüştürmek gerekiyordu. Bu dönüşümün temelini iki süreç oluşturur: örnekleme (sampling) ve kuantizasyon.

Nyquist-Shannon örnekleme teoremi, bir analog sinyalin içerdiği bilginin kaybolmaması için hangi sıklıkta örneklenmesi gerektiğini matematiksel olarak ortaya koydu (Shannon, 1949) [3]. Teoreme göre örnekleme frekansı, sinyaldeki en yüksek frekansın en az iki katı olmalıdır. Örnekleme, sinyalin zaman ekseninde ayrıklaştırılmasıdır. Kuantizasyon ise bu örneklerin genlik (amplitude) ekseninde ayrıklaştırılması işlemidir. Yani, her bir örneğin sürekli genlik değeri, önceden tanımlanmış sonlu sayıdaki seviyeden en yakınına atanır. Bu süreç, İngiliz mühendis Alec Reeves tarafından 1937'de patenti alınan Darbe Kod Modülasyonu (Pulse-Code Modulation, PCM) sisteminin kalbinde yer alıyordu (Reeves, 1938) [4]. PCM, analog sinyali örnekleyerek, kuantize ederek ve her bir kuantize edilmiş değeri bir ikili koda dönüştürerek dijital iletişimde bir devrim oluşturdu. Bell Laboratuvarları'nda 2. Dünya Savaşı sırasında güvenli iletişim için geliştirilen ve daha sonra 1960'larda ticari telefon sistemlerinde kullanılan PCM, bugünkü dijital ses ve CD teknolojisinin temelini oluşturur. Ancak bu süreç, bir miktar bilgi kaybını da beraberinde getirir. Orijinal genlik değeri ile kuantize edilmiş değer arasındaki fark, "kuantizasyon hatası (quantization error)" veya "kuantizasyon gürültüsü (quantization noise)" [5] olarak bilinir ve bu, dijital sinyal işlemenin doğasında var olan bir ödünleşmedir.

### Veri Sıkıştırma Verimlilik Arayışı: Vektör Kuantizasyonu

PCM'de kullanılan skaler kuantizasyon, her bir sinyal örneğini bağımsız olarak ele alır. Ancak 1970'ler ve 80'lerde, özellikle görüntü ve ses sıkıştırma alanlarındaki ilerlemeler, daha verimli bir yaklaşım olan Vektör Kuantizasyonu'nun (VK, VQ) geliştirilmesine yol açtı. VQ, sinyal örneklerini tek tek değil, "vektörler" adı verilen bloklar halinde gruplayarak kuantize eder. Bu yaklaşımın temel mantığı, sinyaldeki komşu örnekler arasındaki korelasyondan faydalanmaktır. Örneğin, bir görüntüdeki komşu pikseller genellikle benzer renk değerlerine sahiptir.

VQ'nun en önemli kilometre taşlarından biri, 1980 yılında Yoseph Linde, Andrés Buzo ve Robert M. Gray tarafından geliştirilen ve isimlerinin baş harfleriyle anılan LBG algoritmasıdır (Linde, Buzo, & Gray, 1980) [6, 7]. LBG algoritması, bir grup eğitim verisinden (örneğin, görüntü vektörleri) yola çıkarak optimum bir "kod sözlüğü" (codebook) oluşturmak için kullanılan yinelemeli bir kümeleme algoritmasıdır. Bu kod sözlüğü, veriyi temsil edecek en iyi prototip vektörleri içerir. Kodlama sırasında, her bir giriş vektörü, kod sözlüğündeki kendisine en yakın (en az bozulmaya neden olan) prototip vektörün indisiyle değiştirilir. Bu sayede, orijinal vektörlere kıyasla çok daha az bit kullanarak veri temsil edilmiş olur. LBG algoritması ve VQ, konuşma tanıma, görüntü sıkıştırma ve desen tanıma gibi birçok alanda temel bir teknik haline gelmiştir.

### **Modern Yapay Zekâ Çağı: Derin Öğrenme Modellerinde Kuantizasyon**

Kuantizasyon kavramı, 21. yüzyılda derin öğrenmenin yükselişiyle birlikte bir kez daha tekâmüle uğradı. Derin sinir ağları (DSA, Deep Neural Networks, DNNs) [8, 9, 11], görüntü tanıma, doğal dil işleme ve otonom sürüş gibi alanlarda olağanüstü başarılar elde etse de bu başarı büyük bir hesaplama maliyetiyle geldi. Milyonlarca, hatta milyarlarca parametreye sâhip bu modeller, genellikle yüksek hassasiyetli 32-bit kayan nokta (FP32) sayılarıyla eğitilir ve çalıştırılır. Bu durum, modellerin büyük bellek ve enerji tüketimine sâhip olmasına neden olur, bu da onları akıllı telefonlar, sensörler ve diğer kenar, uç, edge bilişim cihazları gibi kaynak kısıtlı ortamlarda çalıştırmayı zorlaştırır.

Bu soruna çözüm olarak, sinir ağı kuantizasyonu fikri ortaya çıktı. Buradaki amaç, modelin ağırlıklarını ve/veya aktivasyonlarını FP32'den daha düşük bit genişliğine sahip formatlara, örneğin 8-bit tam sayılara (INT8), 4-bit'e ve hatta 1-bit'e (ikili) dönüştürmektir. Bu, model boyutunu ve bellek bant genişliği gereksinimlerini önemli ölçüde azaltır ve tam sayı aritmetiği, kayan nokta aritmetiğinden çok daha hızlı ve enerji verimli olduğu için çıkarım (inference) hızını artırır.

İlk başta, modeller eğitildikten sonra kuantize ediliyordu (Eğitim Sonrası Kuantizasyon, ESK; Post-Training Quantization - PTQ). Ancak bu yöntem, özellikle düşük bit genişliklerinde model doğruluğunda ciddi düşüslere neden olabiliyordu. Daha sofistike bir yaklaşım olan "Kuantizasyon Farkındalıklı Eğitim, KFE" (Quantization-Aware Training - QAT) ise kuantizasyon işlemini eğitim sürecine dahil eder. QAT sırasında, modelin ileri geçiş (forward pass) adımı ağırlıklar ve aktivasyonlar kuantize edilirken, geri yayılım (backward pass) ve parametre güncellemeleri yüksek hassasiyetli kayan nokta sayılarıyla yapılmaya devam edilir. Bu sayede model, kuantizasyonun neden olacağı hassasiyet kaybını telafi etmeyi öğrenir ve doğrulukta çok az bir kayıpla veya hiç kayıp olmadan önemli ölçüde daha verimli hâle gelir.

## Sonuç

Kuantizasyonun yolculuğu, 20. yüzyılın başlarında evrenin temel yapısını açıklayan teorik bir fizik konsepti olarak başladı. Zamanla, mühendisliğin pratik ihtiyaçlarına cevap vermek için dönüşerek dijital dünyanın temelini attı. Analog sinyalleri dijitalleştirdi, verileri daha verimli sıkıştırılmamızı sağladı ve son olarak, en gelişmiş yapay zekâ modellerini avucumuzun içine sığdıran bir anahtar hâline geldi. Planck'ın atom altı dünyadaki ayrık enerji seviyelerinden, günümüzün derin öğrenme ağlarındaki ayrık sayısal temsillere uzanan bu tekâmül, temel bir fikrin farklı alanlardaki zorlukları çözmek için nasıl uyarlanıp yeniden icâd edilebileceğinin güçlü bir kanıtıdır. Kuantizasyon, soyut bir prensibin somut teknolojiye dönüşme serüveninin en ilham verici örneklerinden biri olarak varlığını sürdürmektedir.

## II. Bellek Kullanımını Azaltmak İçin Bir Strateji Olarak Kuantizasyon

Modern derin öğrenme modelleri, özellikle derin sinir ağları (DNN'ler), görüntü tanıma, doğal dil işleme ve otonom sistemler gibi alanlarda benzeri görülmemiş bir başarı elde etmiştir. Ancak bu başarının bir bedeli vardır: bu modellerin boyutu ve karmaşıklığı, milyonlarca, hatta milyarlarca parametre içerecek şekilde katlanarak artmıştır. Genellikle yüksek hassasiyetli 32-bit kayan nokta (FP32) sayılarıyla temsil edilen bu parametreler (ağırlıklar ve sapmalar), hem depolama (statik bellek) hem de çıkarım sırasındaki çalışma (dinamik bellek veya RAM) için önemli miktarda bellek gerektirir. Bu durum, modellerin akıllı telefonlar, gömülü sistemler ve diğer kenar bilişim cihazları gibi bellek ve güç açısından kısıtlı ortamlara dağıtılmasının önündeki en büyük engellerden birini oluşturmaktadır. Bu zorluğun üstesinden gelmek için geliştirilen en etkili optimizasyon stratejilerinden [44–60] biri kuantizasyondur.

Kuantizasyon, en temel düzeyde, bir sinir ağındaki sayısal değerleri (hem ağırlıkları hem de aktivasyonları) temsil etmek için kullanılan bit sayısını azaltma işlemidir. Bu, sürekli veya yüksek hassasiyetli değerleri daha az sayıda ayrık değere sahip bir aralığa eşleyerek gerçekleştirilir. Derin öğrenme bağlamındaki en yaygın uygulama, standart FP32 formatından 8-bit tam sayı (INT8) formatına geçmektir. Bu dönüşümün bellek üzerindeki etkisi doğrudan ve çarpıcıdır. Her bir parametre 32 bit yerine 8 bit ile temsil edildiğinde, modelin depolama için ihtiyaç duyduğu statik bellek miktarı otomatik olarak dört kat azalır (Jacob et al., 2018) [9]. Örneğin, 400 MB boyutundaki bir FP32 modeli, INT8'e kuantize edildiğinde yaklaşık 100 MB boyutuna düşer. Bu küçülme, modelin depolanmasını, ağ üzerinden iletilmesini ve bir cihaza yüklenmesini önemli ölçüde kolaylaştırır.

Bellek azaltmanın faydaları, statik model boyutunun ötesine uzanır ve çıkarım sırasındaki dinamik bellek kullanımını da kapsar. Bir model çıkarım yaparken, parametrelerin ve ara hesaplama sonuçlarının (aktivasyonlar) işlem birimine (CPU, GPU, TPU vb.) taşınması gerekir. Bu verilerin ana bellekten (DRAM) işlemci önbelleğine (cache) taşınması, genellikle modern hesaplama sistemlerindeki en büyük performans darboğazlarından biridir. Düşük bit genişliğine sahip (örneğin INT8) parametreler ve aktivasyonlar kullanıldığında, aynı miktarda veri transferiyle daha fazla bilgi taşınabilir. Bu, "bellek bant genişliği" üzerindeki baskıyı azaltır. Bellek bant genişliğinin daha verimli kullanılması, daha hızlı çıkarım süreleri ve daha düşük enerji tüketimi anlamına gelir, çünkü veri taşıma işlemi enerji açısından maliyetli bir operasyondur (Horowitz, 2014) [10]. Ayrıca, daha küçük veri türleri, işlemcinin hızlı ancak küçük olan önbelleklerine daha iyi sığar, bu da "önbellek isabet oranını" (cache hit rate) artırarak işlemcinin yavaş olan ana belleğe erişme ihtiyacını azaltır ve performansı daha da artırır.

Ancak bu bellek avantajları, potansiyel bir doğruluk kaybı riskiyle birlikte gelir. Yüksek hassasiyetli değerleri daha düşük hassasiyetli bir formata eşlemek, kaçınılmaz olarak bir miktar bilgi kaybına, yani "kuantizasyon hatasına" yol açar. Bu hatanın yönetilememesi, modelin tahmin doğruluğunda kabul edilemez düşüşlere neden olabilir. Bu sorunu çözmek için iki ana strateji geliştirilmiştir: Eğitim Sonrası Kuantizasyon (PTQ) ve Kuantizasyon Farkındalıklı Eğitim (QAT). PTQ, önceden eğitilmiş bir FP32 modelini alır ve küçük bir kalibrasyon veri seti kullanarak ağırlıkları ve aktivasyonları dönüştürür. Daha basit ve hızlı bir yöntem olmasına rağmen, özellikle 4-bit gibi daha agresif kuantizasyon seviyelerinde doğruluk kaybına daha yatkındır. Öte yandan QAT, kuantizasyon işlemini modelin eğitim sürecine dâhil eder. İleri geçiş sırasında kuantizasyonun neden olduğu hatayı simüle eder, böylece model, eğitim sırasında bu hassasiyet kaybını telafi etmeyi ve buna karşı daha dayanıklı olmayı öğrenir (Jacob et al., 2018) [9]. Bu yaklaşım, genellikle çok az veya hiç doğruluk kaybı olmadan INT8 kuantizasyonuna olanak tanır.

Sonuç olarak kuantizasyon, derin öğrenme modellerinin bellek ayak izini önemli ölçüde azaltmak için vazgeçilmez bir teknik olarak ortaya çıkmıştır. Model boyutunu 4 kat veya daha fazla küçülterek ve bellek bant genişliği verimliliğini artırarak, büyük ve karmaşık modellerin kaynak kısıtlı cihazlarda çalıştırılmasını mümkün kılar. Doğruluk ve verimlilik [44–60] arasındaki bu hassas denge, QAT gibi gelişmiş tekniklerle başarılı bir şekilde yönetildiğinde, kuantizasyon yapay zekânın her yerde ve verimli bir şekilde dağıtılmasının önünü açan temel bir etkinleştirici rolü oynamaktadır.



## Open Science Articles (OSAs)

Bu bağlamdaki "**kanatizasyon, nicemleme, quantization**", makine öğrenimi (machine learning), özellikle derin öğrenme (deep learning) modellerinde kullanılan bir **model optimizasyon tekniğidir**. Amaç, modelin bellek kullanımını ve hesaplama maliyetini azaltmak, ancak doğruluk kaybını minimum tutmaktır.

### Tanım:

**Kuantizasyon (nicemleme, quantization)**, bir modeldeki sayısal değerlerin (genellikle ağırlıklar ve aktivasyonlar) **daha düşük bit derinliğine dönüştürülmesi** işlemidir.

Örneğin:

- 32-bit kayan noktalı sayılar (float32) → 8-bit tam sayılar (int8)
- 16-bit (float16) → 4-bit (int4) gibi

Bu, modelin daha az bellek kaplamasını, daha hızlı çalışmasını ve daha az enerji tüketmesini sağlar.

### Yazılım ve Donanım Açısından Açıklama

#### Yazılım Açısından:

- **Model boyutu küçülür:** 32-bit'ten 8-bit'e geçince model boyutu **4 kat azalır**.
- **Hızlı çıkarım (inference):** Daha küçük sayılarla işlem yapıldığı için matris çarpımları daha hızlı.
- **Düşük doğruluk kaybı:** İyi uygulandığında doğruluk sadece küçük oranda düşer.
- **Kullanılan teknikler:**
  - **Post-training quantization (PTQ):** Eğitilmiş modeli nicemleme.
  - **Quantization-aware training (QAT):** Eğitim sırasında nicemleme etkisini simüle etme.

#### Donanım Açısından:

- **Daha az bellek ihtiyacı:** Daha az RAM ve depolama.
- **Daha az enerji tüketimi:** Mobil cihazlar, IoT, edge cihazlar için ideal.
- **Donanım destekli hızlandırma:** TPU'lar, bazı GPU'lar (NVIDIA TensorRT), ARM NEON gibi SIMD birimleri int8 işlemlerini destekler.
- **Paralel işlem avantajı:** Daha küçük veri tipleri, daha fazla işlemi aynı anda yapabilir.

### Örnek:

- Bir BERT modeli: 400 MB (float32) → 100 MB (int8)
- Mobil uygulamalarda bu, modelin telefona indirilmesini ve çevrimdışı çalışmasını mümkün kılar.

## Kuantum Mekaniğindeki "Kuantizasyon" ile Makine Öğrenimindeki "Kuantizasyon"un Karşılaştırılması

İki kavram aynı kelimeyi kullanıyor ("kantizasyon, quantization") fakat **tamamen farklı alanlarda** ve **farklı anlamlarda**.

| Özellik            | Makine Öğreniminde Quantization                             | Kuantum Mekaniğinde Quantization                                    |
|--------------------|-------------------------------------------------------------|---------------------------------------------------------------------|
| Alan               | Yapay zekâ, bilgisayar bilimi                               | Fizik (özellikle kuantum teorisi)                                   |
| Anlamı             | Sayıların bit derinliğini azaltma (nicemleme)               | Fiziksel büyüklüklerin sürekli değil, ayrık değerler alması         |
| Temel Amaç         | Bellek ve hesaplama verimliliği                             | Doğanın temel davranışını açıklama                                  |
| Örnek              | float32 → int8 dönüşümü                                     | Elektron enerji seviyeleri (örn. hidrojen atomu)                    |
| Matematiksel Temel | Sayısal analiz, doğrusal cebir                              | Lineer cebir, Hilbert uzayları [40, 82–87], operatörler             |
| Ayrıklaştırma      | Veri türlerinin ayrıklaştırılması (örn. 256 değerle temsil) | Enerji, momentum gibi büyüklüklerin kesikli olması                  |
| Fiziksel Gerçeklik | Sanal, yazılımsal bir işlem                                 | Gerçek doğa yasası                                                  |
| Kullanım Yeri      | Derin öğrenme modelleri, mobil AI                           | Atomik ve alt atomik parçacıklar                                    |
| Etkisi             | Model daha küçük, hızlı, verimli                            | Doğanın kuantum davranışını açıklar (örn. tünelleme, süperpozisyon) |
| Benzerlik          | Her ikisinde de "sürekli" → "ayrık" dönüşüm söz konusu      | Her ikisinde de "sürekli" → "ayrık" dönüşüm söz konusu              |
| Farklılık          | Süreklilik, gerçek sayılar → ayrık sayılar (sayısal)        | Süreklilik, klasik fizik → ayrık değerler (fiziksel)                |

Tablo 1: Karşılaştırma

### Ortak Nokta ve Farklılıkların Özeti

#### Ortaklık:

- Her iki kavram da **"sürekli bir sistemi ayrık hâle getirme"** fikrine dayanır.



## *Open Science Articles (OSAs)*

- İngilizce "quantize" kelimesi, "**kesikli hâle getirmek**" anlamına gelir → bu yüzden ikisinde de kullanılır.

### **Farklılıklar:**

- Makine öğrenimindeki kuantizasyon, uygulamalı bir optimizasyon tekniğidir.
- Kuantum mekaniğindeki kuantizasyon, evrenin temel doğasını tanımlayan bir fiziksel gerçekliktir.

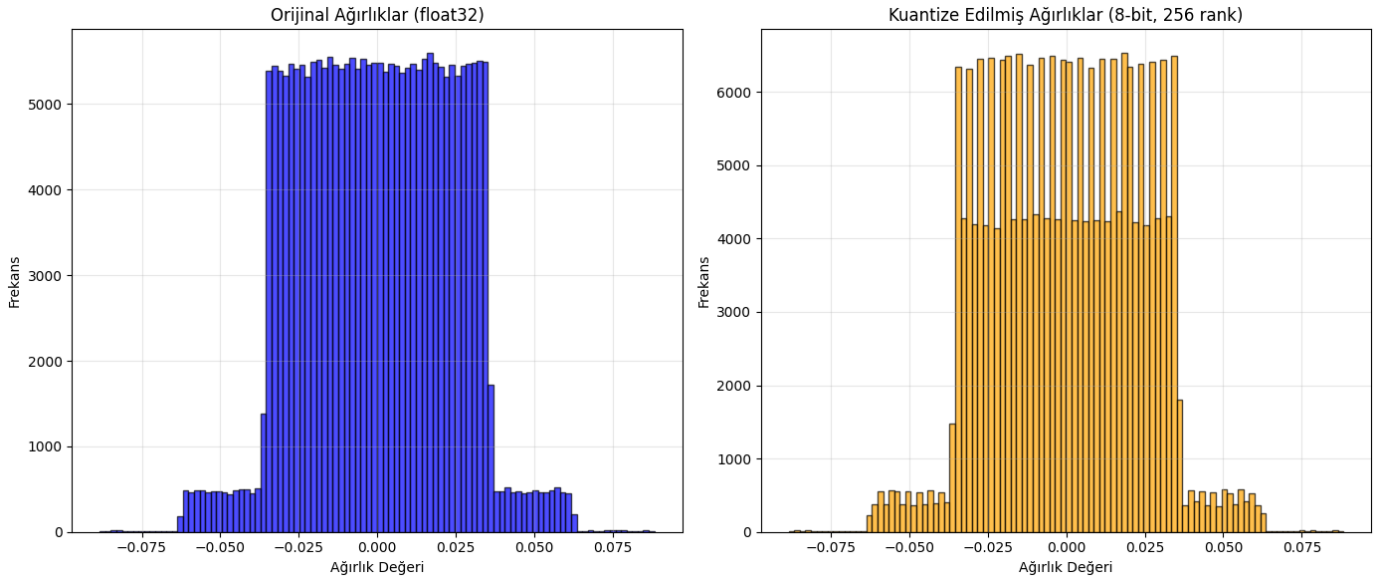
### **Benzetme:**

- Makine öğrenimindeki kuantizasyon → "Bir fotoğrafı 16 milyon renkten 256 renge indirmek" (veri sıkıştırma)
- Kuantum mekaniğindeki kuantizasyon → "Bir merdivenin basamakları var, arada duramazsın" (doğanın kuralı)

### **Sonuç**

- "Quantization to reduce memory"**, özellikle yapay zekâ (YZ, AI) ve kenar hesaplamada (edge computing) **kritik bir optimizasyon yöntemidir**.
- Yazılım ve donanım birlikteliğiyle, cihazlarda hızlı, hafif ve verimli modeller çalıştırılmasını sağlar.
- Kuantum mekaniğindeki "kuantizasyon" ile **kelime benzerliği olsa da, kavramsal olarak farklıdır**.
- Ortak nokta: "**ayrıklaştırma**" fikri. Ancak biri **mühendislik seçimi**, diğeri **doğal yasadır**.

### III. Örnek Uygulamalar



Şekil 1: FPS32 ile INT8 karşılaştırması

--- Orijinal (float32) Ağırlıklar İstatistikleri ---

Min: -0.088387  
Max: 0.088240  
Ortalama: 0.000024  
Std: 0.023631  
Medyan: -0.000017  
Skewness: 0.001167  
Kurtosis: -0.530938

--- Kuantize Edilmiş (8-bit sim) Ağırlıklar İstatistikleri ---

Min: -0.088387  
Max: 0.088240  
Ortalama: 0.000023  
Std: 0.023633  
Medyan: 0.000273  
Skewness: 0.001201  
Kurtosis: -0.531074

--- Bellek Kullanımı ---

Orijinal (float32): 940,584 byte (918.54 KB)  
Kuantize (int8): 235,146 byte (229.63 KB)  
Bellek Tasarrufu: %75.0

--- Kuantizasyon Hataları ---

Mean Squared Error (MSE): 0.00000004  
Mean Absolute Error (MAE): 0.00017329

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import torch
import torch.nn as nn
```

### 1. Basit Bir Sinir Ağı Modeli Oluşturalım

# Rastgele bir model (örneğin, bir görüntü sınıflandırıcı simülasyonu)

```
class SimpleModel(nn.Module):  
    def __init__(self):  
        super(SimpleModel, self).__init__()  
        self.fc1 = nn.Linear(784, 256)  
        self.fc2 = nn.Linear(256, 128)  
        self.fc3 = nn.Linear(128, 10)
```

```
    def forward(self, x):  
        x = torch.relu(self.fc1(x))  
        x = torch.relu(self.fc2(x))  
        x = self.fc3(x)  
        return x
```

# Modeli oluştur

```
model = SimpleModel()
```

### 2. Orijinal Ağırlıkları Al (float32)

# Tüm ağırlıkları bir listeye toplama

```
original_weights = []  
for param in model.parameters():  
    if param.requires_grad:  
        original_weights.extend(param.data.cpu().numpy().flatten())
```

```
original_weights = np.array(original_weights)
```

### 3. 8-Bit (256 Rank) Kuantizasyon Uygula

```
def quantize_to_8bit(weights):  
    # Aralık belirle  
    w_min, w_max = weights.min(), weights.max()  
  
    # 0-255 arası tam sayıya dönüştür (256 farklı değer)  
    quantized = np.round((weights - w_min) / (w_max - w_min) * 255).astype(np.uint8)  
  
    # Tekrar float32'e çevir (simülasyon amaçlı)  
    dequantized = quantized.astype(np.float32) * (w_max - w_min) / 255.0 + w_min  
  
    return dequantized, quantized, w_min, w_max
```

# Kuantize et

```
dequantized_weights, quantized_int8, w_min, w_max = quantize_to_8bit(original_weights)
```

### 4. İstatistiksel Karşılaştırma

```
def print_stats(name, weights):  
    print(f"\n--- {name} Ağırlıklar İstatistikleri ---")  
    print(f"Min: {weights.min():.6f}")  
    print(f"Max: {weights.max():.6f}")  
    print(f"Ortalama: {weights.mean():.6f}")  
    print(f"Std: {weights.std():.6f}")  
    print(f"Medyan: {np.median(weights):.6f}")  
    print(f"Skewness: {stats.skew(weights):.6f}")  
    print(f"Kurtosis: {stats.kurtosis(weights):.6f}")
```

# Orijinal ve kuantize edilmiş istatistikler

```
print_stats("Original (float32)", original_weights)
print_stats("Kuantize Edilmiş (8-bit sim)", dequantized_weights)

### 5. Grafik Çıktıları (Dağılım Karşılaştırması)
plt.figure(figsize=(14, 6))

# 1. Orijinal dağılım
plt.subplot(1, 2, 1)
plt.hist(original_weights, bins=100, color='blue', alpha=0.7, edgecolor='black')
plt.title('Orijinal Ağırlıklar (float32)')
plt.xlabel('Ağırlık Değeri')
plt.ylabel('Frekans')
plt.grid(True, alpha=0.3)

# 2. Kuantize edilmiş dağılım
plt.subplot(1, 2, 2)
plt.hist(dequantized_weights, bins=100, color='orange', alpha=0.7, edgecolor='black')
plt.title('Kuantize Edilmiş Ağırlıklar (8-bit, 256 rank)')
plt.xlabel('Ağırlık Değeri')
plt.ylabel('Frekans')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

### 6. Bellek Kullanımı Karşılaştırması
original_memory = original_weights.nbytes # bytes
quantized_memory = quantized_int8.nbytes # uint8 → 1 byte per element

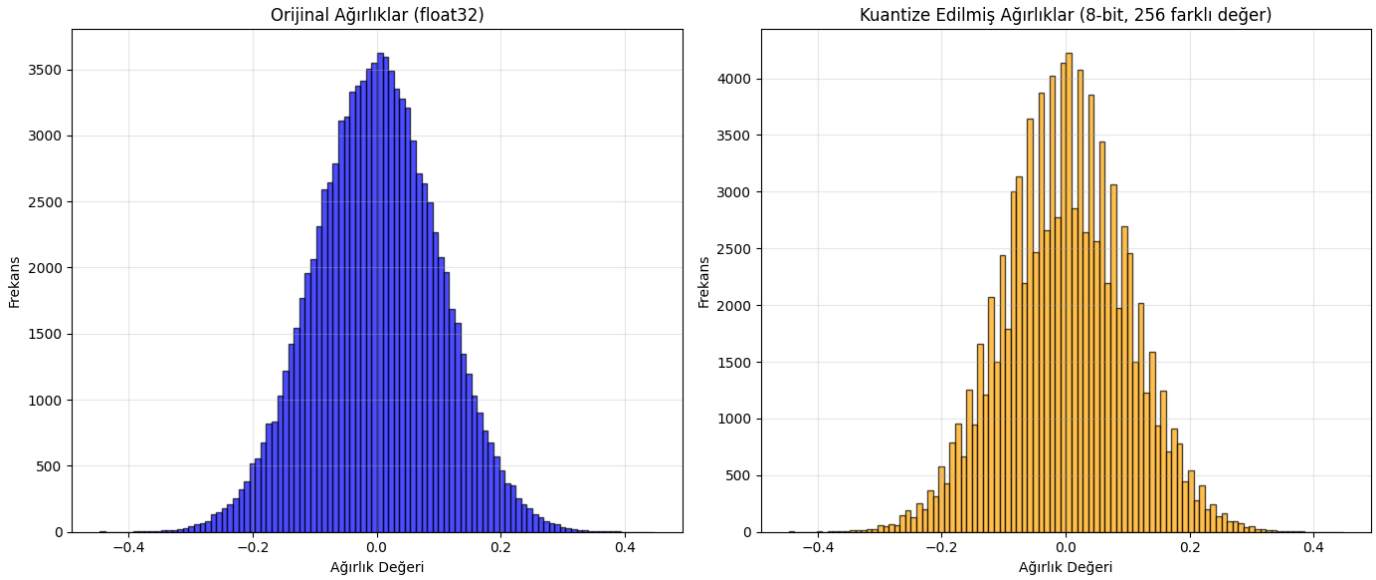
print(f"\n--- Bellek Kullanımı ---")
print(f"Orijinal (float32): {original_memory:,} byte ({original_memory / 1024:.2f} KB)")
print(f"Kuantize (int8): {quantized_memory:,} byte ({quantized_memory / 1024:.2f} KB)")
print(f"Bellek Tasarrufu: %{100 * (1 - quantized_memory / original_memory):.1f}")

### 7. Hata Analizi (Kuantizasyon Kaybı)
mse = np.mean((original_weights - dequantized_weights) ** 2)
mae = np.mean(np.abs(original_weights - dequantized_weights))

print(f"\n--- Kuantizasyon Hataları ---")
print(f"Mean Squared Error (MSE): {mse:.8f}")
print(f"Mean Absolute Error (MAE): {mae:.8f}")
```

Liste 1: FP32 ile INT8 karşılaştırması

Aynı işlemi torch kullanmada yapalım (bâzi kullanıcılarda sorun çıkarabilir)



Şekil 2: FP32 ile INT8 karşılaştırması

--- Orjinal (float32) Ağırlıklar İstatistikleri ---

Min: -0.446560  
Max: 0.447908  
Ortalama: 0.000097  
Std: 0.100090  
Medyan: 0.000265  
Skewness: -0.001760  
Kurtosis: -0.008037

--- Kuantize Edilmiş (8-bit) Ağırlıklar İstatistikleri ---

Min: -0.446560  
Max: 0.447908  
Ortalama: 0.000098  
Std: 0.100104  
Medyan: -0.001080  
Skewness: -0.001955  
Kurtosis: -0.007761

--- Bellek Kullanımı ---

Original (float32): 400,000 byte (390.62 KB)  
Kuantize (int8): 100,000 byte (97.66 KB)  
Bellek Tasarrufu: %75.0

--- Kuantizasyon Hataları ---

Mean Squared Error (MSE): 0.00000102  
Mean Absolute Error (MAE): 0.00087579

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

```
### 1. Rastgele Ağırlık Verisi Üret (Modelden Gelen Gibi)
# Normal dağılımlı ağırlıklar (tipik DNN ağırlıkları gibi)
np.random.seed(42)
```

```
original_weights = np.random.normal(loc=0.0, scale=0.1, size=100000).astype(np.float32)
```

```
# Biraz daha gerçekçi yapmak için sınırlayabiliriz  
original_weights = np.clip(original_weights, -0.5, 0.5)
```

```
### 2. 8-Bit (256 Rank) Kuantizasyon Fonksiyonu
```

```
def quantize_to_8bit(weights):  
    w_min, w_max = weights.min(), weights.max()  
  
    # 0-255 arası tam sayıya dönüştür (8-bit)  
    quantized = np.round((weights - w_min) / (w_max - w_min) * 255).astype(np.uint8)  
  
    # Tekrar float32'e çevir (de-quantize)  
    dequantized = quantized.astype(np.float32) * (w_max - w_min) / 255.0 + w_min  
  
    return dequantized, quantized, w_min, w_max
```

```
# Kuantizasyon uygula
```

```
dequantized_weights, quantized_int8, w_min, w_max = quantize_to_8bit(original_weights)
```

```
### 3. İstatistiksel Karşılaştırma
```

```
def print_stats(name, weights):  
    print(f"\n--- {name} Ağırlıklar İstatistikleri ---")  
    print(f"Min: {weights.min():.6f}")  
    print(f"Max: {weights.max():.6f}")  
    print(f"Ortalama: {weights.mean():.6f}")  
    print(f"Std: {weights.std():.6f}")  
    print(f"Medyan: {np.median(weights):.6f}")  
    print(f"Skewness: {stats.skew(weights):.6f}")  
    print(f"Kurtosis: {stats.kurtosis(weights):.6f}")
```

```
# Karşılaştır
```

```
print_stats("Original (float32)", original_weights)  
print_stats("Kuantize Edilmiş (8-bit)", dequantized_weights)
```

```
### 4. Grafik Çıktıları (Dağılım Karşılaştırması)
```

```
plt.figure(figsize=(14, 6))
```

```
# Orijinal
```

```
plt.subplot(1, 2, 1)  
plt.hist(original_weights, bins=100, color='blue', alpha=0.7, edgecolor='black')  
plt.title('Orijinal Ağırlıklar (float32)')  
plt.xlabel('Ağırlık Değeri')  
plt.ylabel('Frekans')  
plt.grid(True, alpha=0.3)
```

```
# Kuantize edilmiş
```

```
plt.subplot(1, 2, 2)  
plt.hist(dequantized_weights, bins=100, color='orange', alpha=0.7, edgecolor='black')  
plt.title('Kuantize Edilmiş Ağırlıklar (8-bit, 256 farklı değer)')  
plt.xlabel('Ağırlık Değeri')  
plt.ylabel('Frekans')  
plt.grid(True, alpha=0.3)
```

```
plt.tight_layout()  
plt.show()
```



### 5. Bellek Kullanımı Karşılaştırması

```
original_memory = original_weights.nbytes # float32 → 4 byte per element
quantized_memory = quantized_int8.nbytes  # uint8 → 1 byte per element
```

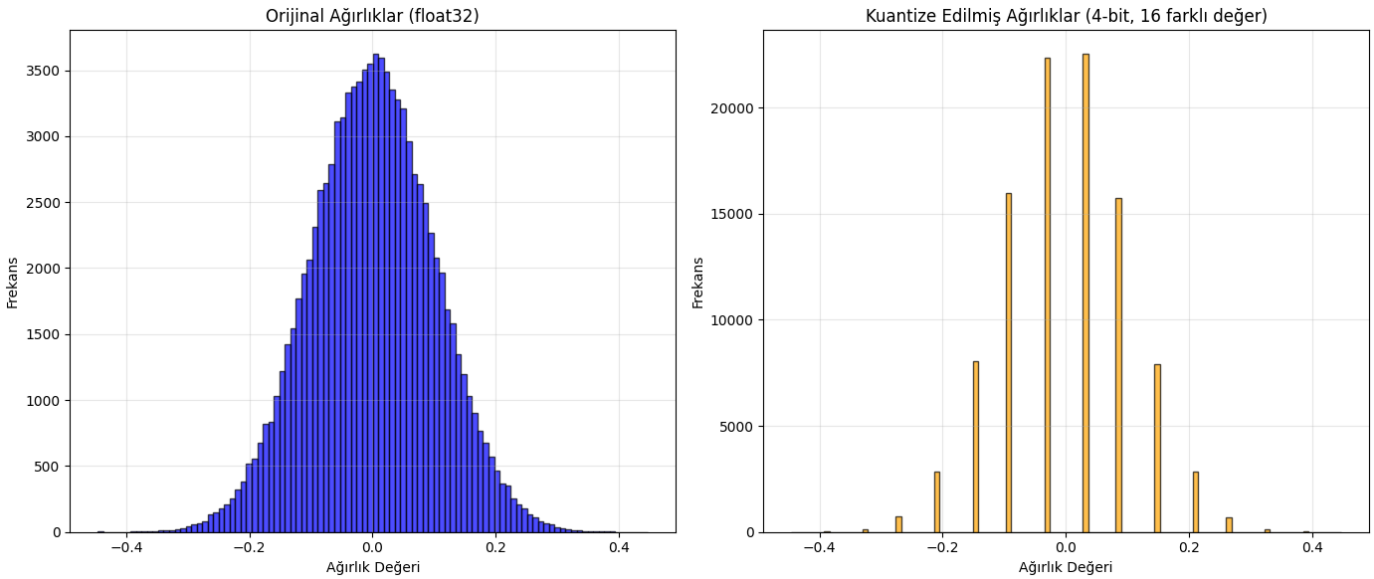
```
print(f"\n--- Bellek Kullanımı ---")
print(f"Original (float32): {original_memory:,} byte ({original_memory / 1024:.2f} KB)")
print(f"Kuantize (int8): {quantized_memory:,} byte ({quantized_memory / 1024:.2f} KB)")
print(f"Bellek Tasarrufu: %{100 * (1 - quantized_memory / original_memory):.1f}%")
```

### 6. Kuantizasyon Hataları

```
mse = np.mean((original_weights - dequantized_weights) ** 2)
mae = np.mean(np.abs(original_weights - dequantized_weights))
```

```
print(f"\n--- Kuantizasyon Hataları ---")
print(f"Mean Squared Error (MSE): {mse:.8f}")
print(f"Mean Absolute Error (MAE): {mae:.8f}")
```

Liste 2: FP32 ile INT8 karşılaştırması



Şekil 3: FP32 ile INT4 karşılaştırması

--- Orijinal (float32) Ağırlıklar İstatistikleri ---

```
Min: -0.446560
Max: 0.447908
Ortalama: 0.000097
Std: 0.100090
Medyan: 0.000265
Skewness: -0.001760
Kurtosis: -0.008037
```

--- Kuantize Edilmiş (4-bit) Ağırlıklar İstatistikleri ---

```
Min: -0.446560
Max: 0.447908
Ortalama: 0.000072
Std: 0.101585
Medyan: -0.029142
Skewness: -0.002881
Kurtosis: -0.000404
```

--- Bellek Kullanımı ---

Original (float32): 400,000 byte (390.62 KB)  
Kuantize (int4): 100,000 byte (97.66 KB)  
Bellek Tasarrufu: %75.0

--- Kuantizasyon Hataları ---

Mean Squared Error (MSE): 0.00029668  
Mean Absolute Error (MAE): 0.01491484

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

```
### 1. Rastgele Ağırlık Verisi Üret (Modelden Gelen Gibi)
# Normal dağılımlı ağırlıklar (tipik DNN ağırlıkları gibi)
np.random.seed(42)
original_weights = np.random.normal(loc=0.0, scale=0.1, size=100000).astype(np.float32)
```

```
# Biraz daha gerçekçi yapmak için sınırlayabiliriz
original_weights = np.clip(original_weights, -0.5, 0.5)
```

```
### 2. 4-Bit (16 Rank) Kuantizasyon Fonksiyonu
# 4-bit (16 farklı değer) için:
def quantize_to_4bit(weights):
    w_min, w_max = weights.min(), weights.max()
    quantized = np.round((weights - w_min) / (w_max - w_min) * 15).astype(np.uint8) # 0-15
    dequantized = quantized * (w_max - w_min) / 15.0 + w_min
    return dequantized, quantized, w_min, w_max
```

```
# Kuantizasyon uygula
#dequantized_weights, quantized_int8, w_min, w_max = quantize_to_4bit(original_weights)
dequantized_weights, quantized_int4, w_min, w_max = quantize_to_4bit(original_weights)
```

```
### 3. İstatistiksel Karşılaştırma
def print_stats(name, weights):
    print(f"\n--- {name} Ağırlıklar İstatistikleri ---")
    print(f"Min: {weights.min():.6f}")
    print(f"Max: {weights.max():.6f}")
    print(f"Ortalama: {weights.mean():.6f}")
    print(f"Std: {weights.std():.6f}")
    print(f"Medyan: {np.median(weights):.6f}")
    print(f"Skewness: {stats.skew(weights):.6f}")
    print(f"Kurtosis: {stats.kurtosis(weights):.6f}")
```

```
# Karşılaştır
print_stats("Original (float32)", original_weights)
print_stats("Kuantize Edilmiş (4-bit)", dequantized_weights)
```

```
### 4. Grafik Çıktıları (Dağılım Karşılaştırması)
plt.figure(figsize=(14, 6))
```

```
# Original
plt.subplot(1, 2, 1)
plt.hist(original_weights, bins=100, color='blue', alpha=0.7, edgecolor='black')
plt.title('Original Ağırlıklar (float32)')
plt.xlabel('Ağırlık Değeri')
```

```
plt.ylabel('Frekans')
plt.grid(True, alpha=0.3)

# Kuantize edilmiş
plt.subplot(1, 2, 2)
plt.hist(dequantized_weights, bins=100, color='orange', alpha=0.7, edgecolor='black')
plt.title('Kuantize Edilmiş Ağırlıklar (4-bit, 16 farklı değer)')
plt.xlabel('Ağırlık Değeri')
plt.ylabel('Frekans')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

### 5. Bellek Kullanımı Karşılaştırması
original_memory = original_weights.nbytes # float32 → 4 byte per element
quantized_memory = quantized_int4.nbytes # uint8 → 1 byte per element

print(f"\n--- Bellek Kullanımı ---")
print(f"Original (float32): {original_memory:,} byte ({original_memory / 1024:.2f} KB)")
print(f"Kuantize (int4): {quantized_memory:,} byte ({quantized_memory / 1024:.2f} KB)")
print(f"Bellek Tasarrufu: %{100 * (1 - quantized_memory / original_memory):.1f}")

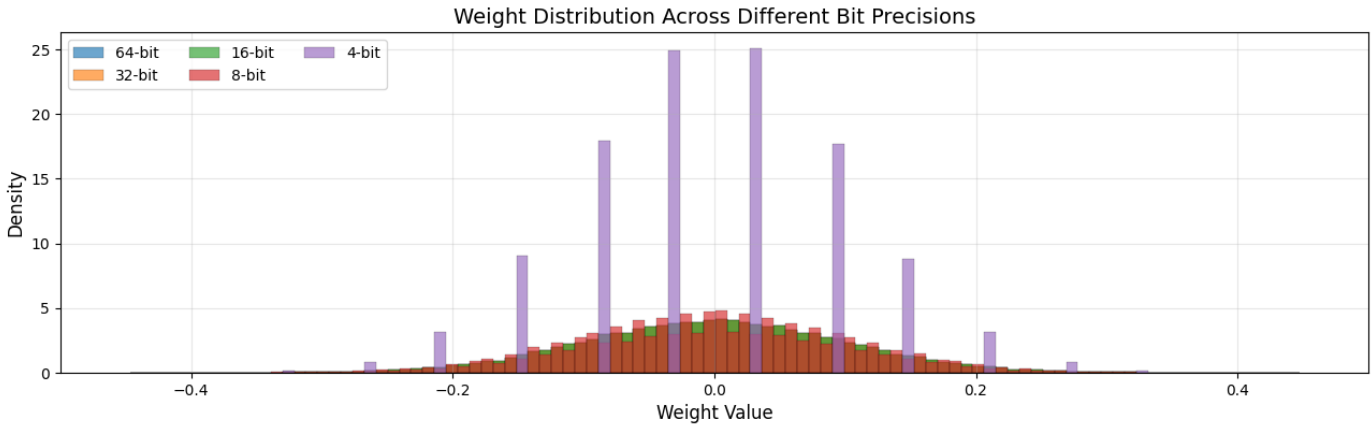
### 6. Kuantizasyon Hataları
mse = np.mean((original_weights - dequantized_weights)** 2)
mae = np.mean(np.abs(original_weights - dequantized_weights))

print(f"\n--- Kuantizasyon Hataları ---")
print(f"Mean Squared Error (MSE): {mse:.8f}")
print(f"Mean Absolute Error (MAE): {mae:.8f}")
```

Liste 2: FP32 ile INT4 karşılaştırması

**BitFields Comparison: Quantization Impact on Neural Network Weights**  
**Statistical Summary and Weight Distribution**

| Bit | Min       | Max      | Mean      | Std      | MSE      | MAE      | Memory (KB) | Compression |
|-----|-----------|----------|-----------|----------|----------|----------|-------------|-------------|
| 64  | -0.446560 | 0.447908 | -0.000042 | 0.100016 | 0.00e+00 | 0.00e+00 | 390.62      | 1.00x       |
| 32  | -0.446560 | 0.447908 | -0.000042 | 0.100016 | 6.45e-18 | 1.72e-09 | 195.31      | 2.00x       |
| 16  | -0.446560 | 0.447908 | -0.000042 | 0.100016 | 1.55e-11 | 3.41e-06 | 390.62      | 1.00x       |
| 8   | -0.446560 | 0.447908 | -0.000040 | 0.100027 | 1.02e-06 | 8.76e-04 | 390.62      | 1.00x       |
| 4   | -0.446560 | 0.447908 | -0.000044 | 0.101594 | 2.97e-04 | 1.49e-02 | 390.62      | 1.00x       |



Şekil 4: 4–64 bit karşılaştırması

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Rastgele ağırlık verisi (DNN ağırlığı simülasyonu)
np.random.seed(42)
original_weights = np.random.normal(loc=0.0, scale=0.1, size=50000).astype(np.float64)
original_weights = np.clip(original_weights, -0.5, 0.5)

# Kuantizasyon fonksiyonu
def quantize_weights(weights, bits):
    if bits == 64:
        return weights.astype(np.float64)
    elif bits == 32:
        return weights.astype(np.float32)
    elif bits == 16:
        w_min, w_max = weights.min(), weights.max()
        quantized = np.round((weights - w_min) / (w_max - w_min) * 65535).astype(np.uint16)
        return quantized.astype(np.float32) * (w_max - w_min) / 65535.0 + w_min
    elif bits == 8:
        w_min, w_max = weights.min(), weights.max()
```

```
quantized = np.round((weights - w_min) / (w_max - w_min) * 255).astype(np.uint8)
return quantized.astype(np.float32) * (w_max - w_min) / 255.0 + w_min
elif bits == 4:
    w_min, w_max = weights.min(), weights.max()
    quantized = np.round((weights - w_min) / (w_max - w_min) * 15).astype(np.uint8)
    return quantized.astype(np.float32) * (w_max - w_min) / 15.0 + w_min
else:
    raise ValueError("Unsupported bit depth")

# Tüm bit seviyeleri için kuantize et
bit_configs = [64, 32, 16, 8, 4]
quantized_weights_list = [quantize_weights(original_weights, b) for b in bit_configs]

# İstatistikleri hesapla
stats_data = []
colors = ['tab:blue', 'tab:orange', 'tab:green', 'tab:red', 'tab:purple']
labels = [f'{b}-bit' for b in bit_configs]

for i, weights in enumerate(quantized_weights_list):
    b = bit_configs[i]
    mse = np.mean((original_weights - weights) ** 2) if b != 64 else 0.0
    mae = np.mean(np.abs(original_weights - weights)) if b != 64 else 0.0
    memory_usage = weights.nbytes
    compression_ratio = original_weights.nbytes / memory_usage

    stats_data.append({
        'Bit': b,
        'Min': f"{weights.min():.6f}",
        'Max': f"{weights.max():.6f}",
        'Mean': f"{weights.mean():.6f}",
        'Std': f"{weights.std():.6f}",
        'MSE': f"{mse:.2e}",
        'MAE': f"{mae:.2e}",
        'Memory (KB)': f"{memory_usage / 1024:.2f}",
        'Compression': f"{compression_ratio:.2f}x"
    })

# === GÖRSEL: Tablo + Grafik (Düzenli Hizalı) ===
fig = plt.figure(figsize=(14, 10))

# Başlık
plt.suptitle("BitFields Comparison: Quantization Impact on Neural Network Weights\n"
            "Statistical Summary and Weight Distribution",
            fontsize=16, fontweight='bold', y=0.98)

# --- TABLO (Üst kısım) ---
ax_table = fig.add_subplot(2, 1, 1)
ax_table.axis('tight')
ax_table.axis('off')

collabels = list(stats_data[0].keys())
row_data = [[row[key] for key in collabels] for row in stats_data]

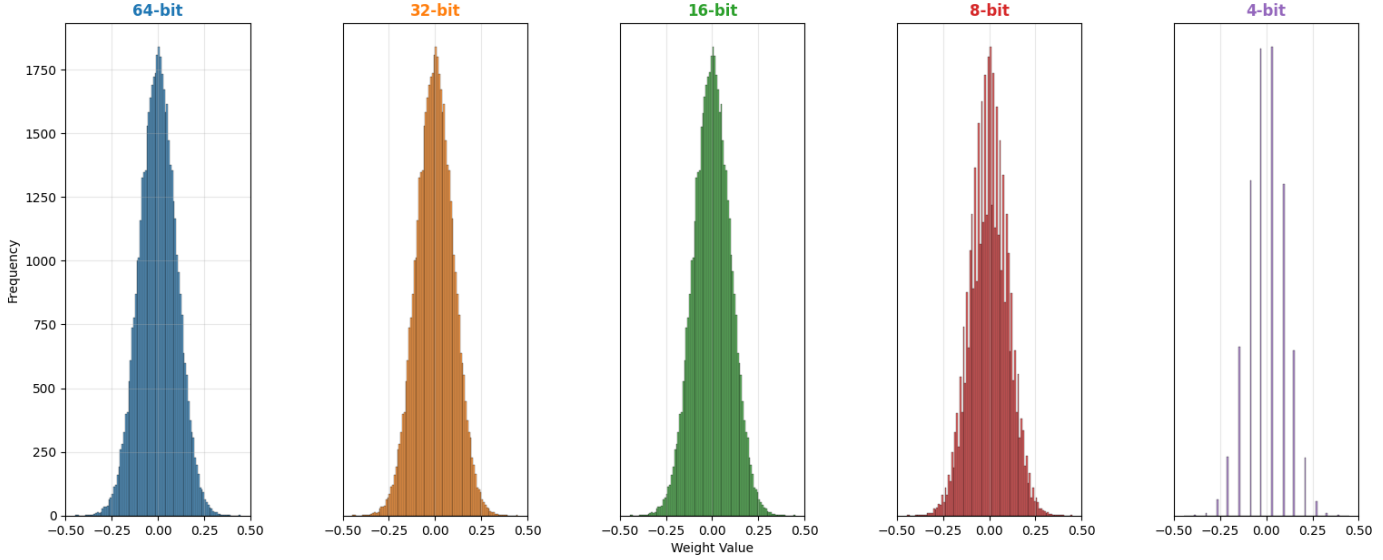
table = ax_table.table(cellText=row_data,
                      collabels=collabels,
                      cellLoc='center',
                      colColours=['#f2f2f2'] * len(collabels),
```

```
loc='center',  
bbox=[0.05, 0.1, 0.9, 0.8]) # Sol, alt, genişlik, yükseklik
```

```
table.auto_set_font_size(False)  
table.set_fontsize(9)  
table.scale(1, 2.2) # Yatay ve dikey ölçek  
  
# Tablo başlığını kalın yap (opsiyonel)  
for (i, key) in enumerate(collabels):  
    table[(0, i)].set_facecolor('#4472C4') # Mavi başlık  
    table[(0, i)].set_text_props(color='white', weight='bold')  
  
# --- GRAFİK (Alt kısım) ---  
ax_plot = fig.add_subplot(2, 1, 2)  
ax_plot.set_xlim(-0.5, 0.5)  
  
for i, weights in enumerate(quantized_weights_list):  
    ax_plot.hist(weights, bins=100, alpha=0.65, color=colors[i],  
                label=f'{bit_configs[i]}-bit', density=True,  
                edgecolor='black', linewidth=0.2)  
  
ax_plot.set_xlabel('Weight Value', fontsize=12)  
ax_plot.set_ylabel('Density', fontsize=12)  
ax_plot.set_title('Weight Distribution Across Different Bit Precisions', fontsize=14)  
ax_plot.legend(loc='upper left', ncol=3, fontsize=10)  
ax_plot.grid(True, alpha=0.3)  
  
# Genel boşluk ayarı: Tablo ile grafik arasında yeterli mesafe  
plt.subplots_adjust(left=0.08, right=0.95, top=0.88, bottom=0.12, hspace=0.4)  
  
plt.show()
```

Liste 4: 4–64 bit karşılaştırması

| Bit | Min       | Max      | Mean      | Std      | MSE      | MAE      | Memory (KB) | Compression |
|-----|-----------|----------|-----------|----------|----------|----------|-------------|-------------|
| 64  | -0.446560 | 0.447908 | -0.000042 | 0.100016 | 0.00e+00 | 0.00e+00 | 390.62      | 1.00x       |
| 32  | -0.446560 | 0.447908 | -0.000042 | 0.100016 | 6.45e-18 | 1.72e-09 | 195.31      | 2.00x       |
| 16  | -0.446560 | 0.447908 | -0.000042 | 0.100016 | 1.55e-11 | 3.41e-06 | 390.62      | 1.00x       |
| 8   | -0.446560 | 0.447908 | -0.000040 | 0.100027 | 1.02e-06 | 8.76e-04 | 390.62      | 1.00x       |
| 4   | -0.446560 | 0.447908 | -0.000044 | 0.101594 | 2.97e-04 | 1.49e-02 | 390.62      | 1.00x       |



Tablo 5: 4–64 bit karşılaştırması

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Rastgele ağırlık verisi
np.random.seed(42)
original_weights = np.random.normal(loc=0.0, scale=0.1, size=50000).astype(np.float64)
original_weights = np.clip(original_weights, -0.5, 0.5)

# Kuantizasyon fonksiyonu
def quantize_weights(weights, bits):
    if bits == 64:
        return weights.astype(np.float64)
    elif bits == 32:
        return weights.astype(np.float32)
    elif bits == 16:
        w_min, w_max = weights.min(), weights.max()
        quantized = np.round((weights - w_min) / (w_max - w_min) * 65535).astype(np.uint16)
        return quantized.astype(np.float32) * (w_max - w_min) / 65535.0 + w_min
    elif bits == 8:
        w_min, w_max = weights.min(), weights.max()
        quantized = np.round((weights - w_min) / (w_max - w_min) * 255).astype(np.uint8)
        return quantized.astype(np.float32) * (w_max - w_min) / 255.0 + w_min
    elif bits == 4:
        w_min, w_max = weights.min(), weights.max()
        quantized = np.round((weights - w_min) / (w_max - w_min) * 15).astype(np.uint8)
```



```
        return quantized.astype(np.float32) * (w_max - w_min) / 15.0 + w_min
    else:
        raise ValueError("Unsupported bit depth")

# Bit yapıları
bit_configs = [64, 32, 16, 8, 4]
quantized_weights_list = [quantize_weights(original_weights, b) for b in bit_configs]
colors = ['tab:blue', 'tab:orange', 'tab:green', 'tab:red', 'tab:purple']

# İstatistikler
stats_data = []
for i, weights in enumerate(quantized_weights_list):
    b = bit_configs[i]
    mse = np.mean((original_weights - weights) ** 2) if b != 64 else 0.0
    mae = np.mean(np.abs(original_weights - weights)) if b != 64 else 0.0
    memory_usage = weights.nbytes
    compression_ratio = original_weights.nbytes / memory_usage

    stats_data.append({
        'Bit': b,
        'Min': f"{weights.min():.6f}",
        'Max': f"{weights.max():.6f}",
        'Mean': f"{weights.mean():.6f}",
        'Std': f"{weights.std():.6f}",
        'MSE': f"{mse:.2e}",
        'MAE': f"{mae:.2e}",
        'Memory (KB)': f"{memory_usage / 1024:.2f}",
        'Compression': f"{compression_ratio:.2f}x"
    })

# === GÖRSEL: Tablo + Ayrı Grafikler ===
fig = plt.figure(figsize=(16, 10)) # <--- Burada figsize belirtilir

# Başlık
plt.suptitle("BitFields Analysis: Quantization Effects on Neural Network Weights\n"
            "Statistical Summary and Individual Weight Distributions",
            fontsize=16, fontweight='bold', y=0.96)

# --- 1. SATIR: Tablo ---
ax_table = fig.add_subplot(3, 1, 1)
ax_table.axis('tight')
ax_table.axis('off')

collabels = list(stats_data[0].keys())
row_data = [[row[key] for key in collabels] for row in stats_data]

table = ax_table.table(cellText=row_data,
                      collabels=collabels,
                      cellLoc='center',
                      colColours=['#f2f2f2'] * len(collabels),
                      loc='center',
                      bbox=[0.05, 0.2, 0.9, 0.6])

table.auto_set_font_size(False)
table.set_fontsize(9)
table.scale(1, 2)
```

```
# Başlık satırını renkli
for i in range(len(colLabels)):
    table[(0, i)].set_facecolor('#4472C4')
    table[(0, i)].set_text_props(color='white', weight='bold')

# --- 2. ve 3. SATIR: 5 Ayrı Grafik (2 satır, 5 sütun) ---
# figsize yukarıda fig'de verildi, artık subplots'a vermeye gerek yok
axes = fig.subplots(2, 5, gridspec_kw={'width_ratios': [1, 1, 1, 1, 1],
                                       'height_ratios': [0.2, 1]})

# Üst satırını temizle (boş)
for ax in axes[0]:
    ax.axis('off')

# Alt satır: Dağılımlar
for i in range(5):
    ax = axes[1][i]
    weights = quantized_weights_list[i]
    bit = bit_configs[i]

    ax.hist(weights, bins=100, color=colors[i], alpha=0.7, edgecolor='black', linewidth=0.3)
    ax.set_title(f'{bit}-bit', fontsize=12, fontweight='bold', color=colors[i])
    ax.set_xlim(-0.5, 0.5)

    if i == 0:
        ax.set_ylabel('Frequency', fontsize=10)
    else:
        ax.set_yticks([])

    if i == 2:
        ax.set_xlabel('Weight Value', fontsize=10)

    ax.grid(True, alpha=0.3)

# Düzenleme
plt.subplots_adjust(left=0.06, right=0.95, top=0.88, bottom=0.1, hspace=0.4, wspace=0.5)

plt.show()
```

Liste 5: 4–64 bit karşılaştırması

#### Gözlemlenecekler:

- **64/32-bit:** Pürüzsüz, normal dağılım
- **16-bit:** Hafif bantlaşma başlar
- **8-bit:** Açıkça 256 değer → dikey çizgiler gibi
- **4-bit:** Sadece 16 değer → 15 dikey "sütun" şeklinde görünür

**4-bit grafiğinde,** ağırlıkların sadece 16 farklı değeri vardır: **merdiven etkisi** çok belirgin!

#### IV. Kuantum Hesaplamalarda Kuantizasyonun 3 Temel Yeri

##### 1. Fiziksel Kuantizasyon (Kuantum Mekanikinin Temeli)

Kuantum hesaplama, doğrudan kuantum mekaniğinin kuantize doğasına dayanır.

- Enerji seviyeleri ayrık,
- Spin, momentum, açısal momentum gibi büyüklükler kuantizedir (nicemlenmiştir),
- Kübitler bu ayrık durumların süperpozisyonuyla çalışır.

Örnek:

- Bir elektronun spin değeri sadece  $+\frac{1}{2}$  veya  $-\frac{1}{2}$  olabilir  $\rightarrow$  kuantize edilmiş.
- Bir kuantum noktada (quantum dot) elektronun enerji seviyesi ayrık (örneğin  $E_0, E_1, E_2$ ).

Bu kuantizasyonun fiziksel seviyesi  $\rightarrow$  kuantum bilgisayarın çalışma prensibinin temeli.

##### 2. Veri Kuantizasyonu (Quantum Machine Learning - QML)

Klasik verileri (görüntü, ses, sayısal veri) kuantum devrelerine sokmak için, verilerin kuantum durumlarına eşlenmesi gerekir. Bu eşleme sırasında veri kuantizasyonu yapılır.

- Klasik veri (örneğin 32-bit float)  $\rightarrow$  kuantum devresinde parametre olarak kullanılır.
- Bu parametreler, parametrik kuantum devrelerinde (variational quantum circuits) döner kapılar (örneğin  $R_y(\theta)$ ) için açılar olarak girer.
- Bu açılar sınırlı çözünürlükle temsil edilir  $\rightarrow$  kuantizasyon zorunludur.

Örnek: Genlik Kodlama (Amplitude Encoding)

- 4 boyutlu vektör:  $[0.1, 0.3, 0.5, 0.1] \rightarrow$  2 kübitlik bir duruma:  $[\psi] = 0.1|00\rangle + 0.3|01\rangle + 0.5|10\rangle + 0.1|11\rangle$  veya  $[\psi] = 0.1|00\rangle + 0.3|01\rangle + 0.5|10\rangle + 0.1|11\rangle$
- Ama bu katsayılar (amplitüdler) sınırlı duyarlılıkla temsil edilir  $\rightarrow$  kuantizasyon hatası oluşur.

Etki:

- Düşük bit derinliği  $\rightarrow$  yüksek hata  $\rightarrow$  algoritma doğruluğu düşer.
- Bu yüzden QML’de veri kuantizasyonu, modelin genelleme kapasitesini etkiler.

### 3. Kuantum Devre Parametrelerinin Kuantizasyonu

Kuantum devrelerinde kullanılan döner kapılar (rotation gates), açısal parametrelerle çalışır:  $R_x(\theta)$ ,  $R_y(\theta)$ ,  $R_z(\theta)$

Bu açılar ( $\theta$ ), donanımda sınırlı çözünürlükle uygulanır.

Örnek:

- Teoride:  $\theta = 1.23456789$  rad
- Gerçek donanımda: sadece 8-bit hassasiyet  $\rightarrow$  256 farklı açı  $\rightarrow \theta \approx 1.234$  rad

Bu, parametre kuantizasyonu olarak adlandırılır.

Etkileri:

- QAOA, VQE gibi varyasyonel algoritmalarda optimum noktaya ulaşmayı geciktirir.
- Robust olmayan çözümler oluşabilir.
- Hata yayılımı (error propagation) artar.

Kuantizasyonun QML’de Pratik Örneği

# Klasik veri

$x = 0.123456789$  # float64

# 8-bit kuantizasyon

$x_{\text{quantized}} = \text{round}(x * 255) / 255$  #  $0.123456 \rightarrow 0.123...$

# Kuantum devresine giriş

#  $|\psi\rangle = \cos(x_{\text{quantized}})|0\rangle + \sin(x_{\text{quantized}})|1\rangle$

Bu kuantizasyon, girdi hatası yaratır ve ileri beslemeyle sonuç doğruluğunu etkiler.

### 4. Kuantum Hesaplamalarda Kuantizasyonun Avantajları ve Riskleri

| Özellik    | Açıklama                                                                                                                                                         |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Avantajlar | <ul style="list-style-type: none"><li>- Daha az hafıza ihtiyacı</li><li>- Daha hızlı simülasyon</li><li>- Donanım uyumluluğu (sınırlı DAC çözünürlüğü)</li></ul> |
| Riskler    | <ul style="list-style-type: none"><li>- Bilgi kaybı</li><li>- Algoritma sapması</li><li>- Yakınsama sorunları (QAOA, VQE)</li></ul>                              |

|                 |                                                                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------|
| <b>Çözümler</b> | - Hata düzeltme<br>- Daha yüksek bit derinliği (12-16 bit)<br>- Kuantizasyon-farklılaştırılmış eğitim |
|-----------------|-------------------------------------------------------------------------------------------------------|

Tablo 6: Kuantum Hesaplamalarda Kuantizasyonun Avantajları ve Riskleri

## 5. Kuantum Hesaplama ile Klasik Kuantizasyon Karşılaştırması

| Özellik                   | Klasik Kuantizasyon (AI)    | Kuantum Hesaplamadaki Kuantizasyon      |
|---------------------------|-----------------------------|-----------------------------------------|
| <b>Amaç</b>               | Bellek ve hız optimizasyonu | Veri giriş uyumu, donanım sınırlamaları |
| <b>Uygulama</b>           | Ağırlıklar: float32 → int8  | Parametreler: float64 → 8-12 bit aç     |
| <b>Etki</b>               | Model hızlanır, hafifler    | Algoritma doğruluğu düşer               |
| <b>Temel</b>              | Sayısal analiz              | Kuantum fiziği + sinyal işleme          |
| <b>Fiziksel Gerçeklik</b> | Sanal (yazılımsal)          | Gerçek (donanım DAC/ADC sınırları)      |

Tablo 7: Kuantum Hesaplama ile Klasik Kuantizasyon Karşılaştırması

## 6. Gelecek: Kuantum Kuantizasyonu mu?

Bazı araştırmalar, kuantum sistemlerin kendi içinde veri kuantizasyonu yapabileceğini öne sürüyor:

- Kuantum ADC (Analog-to-Digital Converter): Klasik sinyali doğrudan kuantum durumuna çevirirken kuantize eder.
- Kuantum NIC (Sinirsel/Nöral Arayüz Çipi/Yongası, Neural Interface Chip): Beyin sinyallerini kuantum devrelere sokarken kuantizasyon yapar.

Bu alanlar henüz erken aşamada, ama kuantum-nöromorfik bilişimde önemli olacaktır.

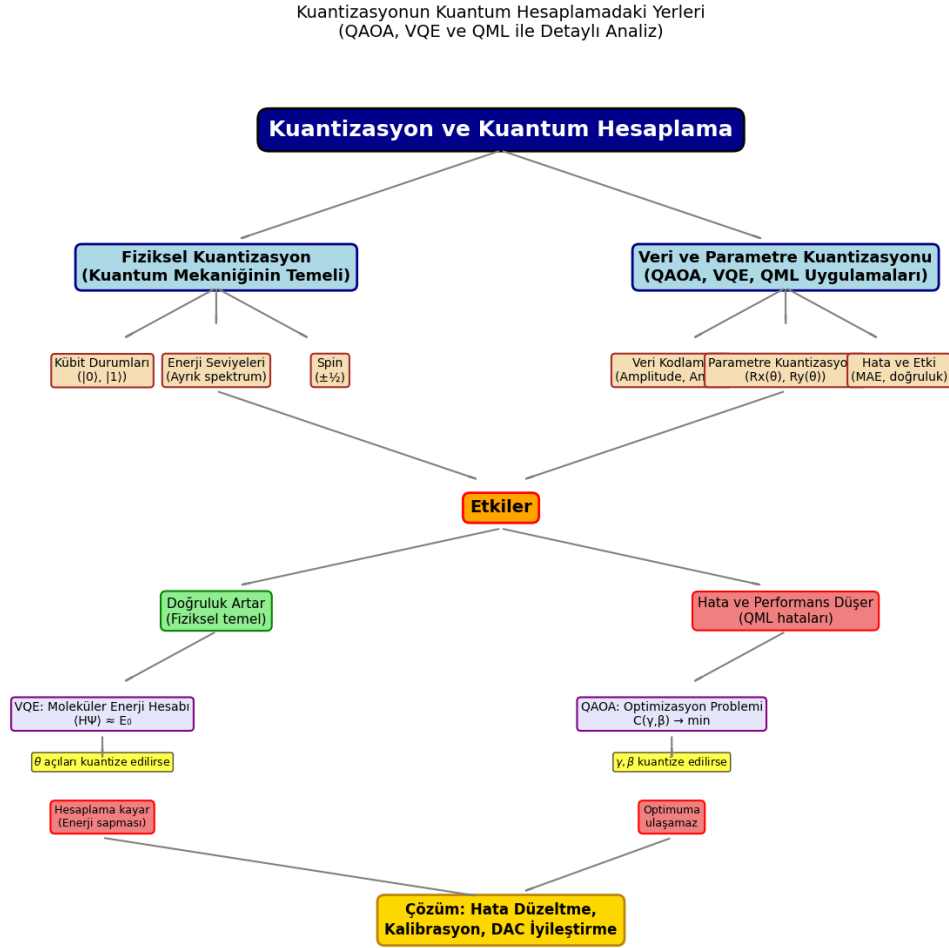
## Sonuç: Kuantizasyon, Kuantum Hesaplama Nasıl Kullanılır?

| Kullanım Yeri             | Açıklama                                                  |
|---------------------------|-----------------------------------------------------------|
| 1. Fiziksel Temel         | Kübitlerin doğası kuantize → enerji, spin, durumlar ayrık |
| 2. Veri Girişinde         | Klasik veri → kuantum durumuna eşlenirken kuantize edilir |
| 3. Devre Parametrelerinde | Dönme açıları sınırlı çözünürlükle uygulanır              |
| 4. Hata ve Optimizasyonda | Kuantizasyon hatası QML'de kritik, kontrol edilmeli       |

Tablo 8: Kuantizasyon, Kuantum Hesaplama Nasıl Kullanılır?

Kuantum hesaplama [26–43], kuantizasyon olmadan var olamaz, çünkü temeli zaten kuantize bir fiziktir. Ancak, klasik veri işlemedeki gibi veri kuantizasyonu da performans ve doğruluk açısından dikkatli yönetilmelidir.

## V. Kuantizasyonu Akış Diyagramı Olarak Gösterme



Şekil 5: Kuantizasyonun Kuantum Hesaplamadaki Yerleri

```
import matplotlib.pyplot as plt
```

```
# Figure ayarları
fig, ax = plt.subplots(figsize=(16, 12))
ax.set_xlim(0, 12)
ax.set_ylim(0, 14)
ax.axis('off') # Eksenleri gizle
```

```
# Stil tanımları
title_style = dict(boxstyle="round,pad=0.5", facecolor="darkblue", edgecolor="black",
linewidth=2)
```

```
main_box = dict(boxstyle="round,pad=0.4", facecolor="lightblue", edgecolor="navy",
linewidth=2)
data_box = dict(boxstyle="round,pad=0.3", facecolor="wheat", edgecolor="brown",
linewidth=1.5)
effect_box = dict(boxstyle="round,pad=0.4", facecolor="orange", edgecolor="red", linewidth=2)
pos_box = dict(boxstyle="round,pad=0.4", facecolor="lightgreen", edgecolor="green",
linewidth=1.5)
neg_box = dict(boxstyle="round,pad=0.4", facecolor="lightcoral", edgecolor="red",
linewidth=1.5)
detail_box = dict(boxstyle="round,pad=0.3", facecolor="lavender", edgecolor="purple",
linewidth=1.5)

# === 1. ANA BAŞLIK ===
ax.annotate('Kuantizasyon ve Kuantum Hesaplama',
            xy=(6, 13), xytext=(6, 13),
            ha='center', va='center',
            fontsize=18, fontweight='bold',
            color='white', # <-- metin rengi
            bbox=title_style)

# === 2. İKİ ANA DAL ===
ax.annotate('Fiziksel Kuantizasyon\n(Kuantum Mekanığının Temeli)',
            xy=(3.5, 11), xytext=(3.5, 11),
            ha='center', va='center', fontsize=13, fontweight='bold',
            bbox=main_box)

ax.annotate('Veri ve Parametre Kuantizasyonu\n(QAOA, VQE, QML Uygulamaları)',
            xy=(8.5, 11), xytext=(8.5, 11),
            ha='center', va='center', fontsize=13, fontweight='bold',
            bbox=main_box)

# === 3. FİZİKSEL ALT KUTULAR ===
phys_y = 9.5
ax.annotate('Qubit Durumları\n(|0>, |1>)', xy=(2.5, phys_y), ha='center', va='center',
            fontsize=10, bbox=data_box)
ax.annotate('Enerji Seviyeleri\n(Ayrık spektrum)', xy=(3.5, phys_y), ha='center',
            va='center', fontsize=10, bbox=data_box)
ax.annotate('Spin\n(±½)', xy=(4.5, phys_y), ha='center', va='center', fontsize=10,
            bbox=data_box)

# === 4. VERİ ALT KUTULAR ===
data_y = 9.5
ax.annotate('Veri Kodlama\n(Amplitude, Angle)', xy=(7.5, data_y), ha='center', va='center',
            fontsize=10, bbox=data_box)
ax.annotate('Parametre Kuantizasyonu\n(Rx(θ), Ry(θ))', xy=(8.5, data_y), ha='center',
            va='center', fontsize=10, bbox=data_box)
ax.annotate('Hata ve Etki\n(MAE, doğruluk)', xy=(9.5, data_y), ha='center', va='center',
            fontsize=10, bbox=data_box)

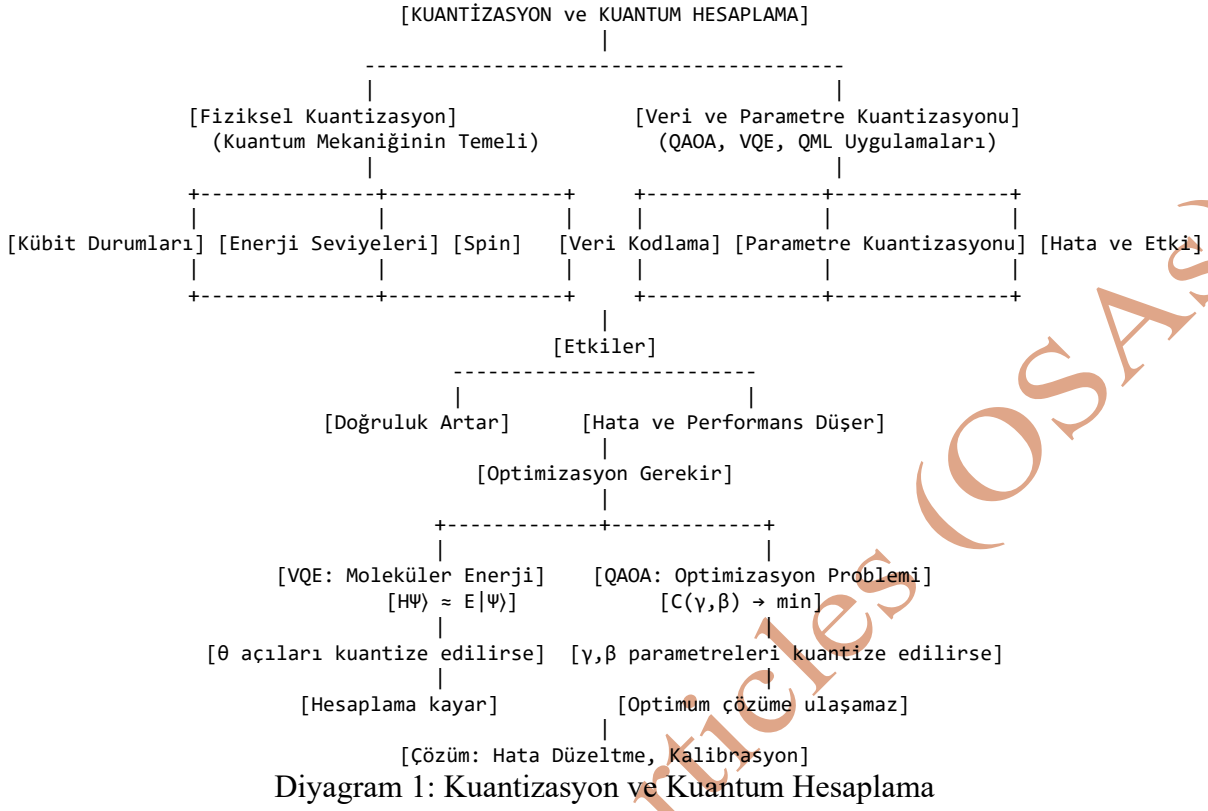
# === 5. ETKİLER ===
ax.annotate('Etkiler', xy=(6, 7.5), ha='center', va='center', fontsize=14, fontweight='bold',
            bbox=effect_box)

# === 6. POZİTİF / NEGATİF ===
ax.annotate('Doğruluk Artar\n(Fiziksel temel)', xy=(3.5, 6), ha='center', va='center',
            fontsize=11, bbox=pos_box)
```

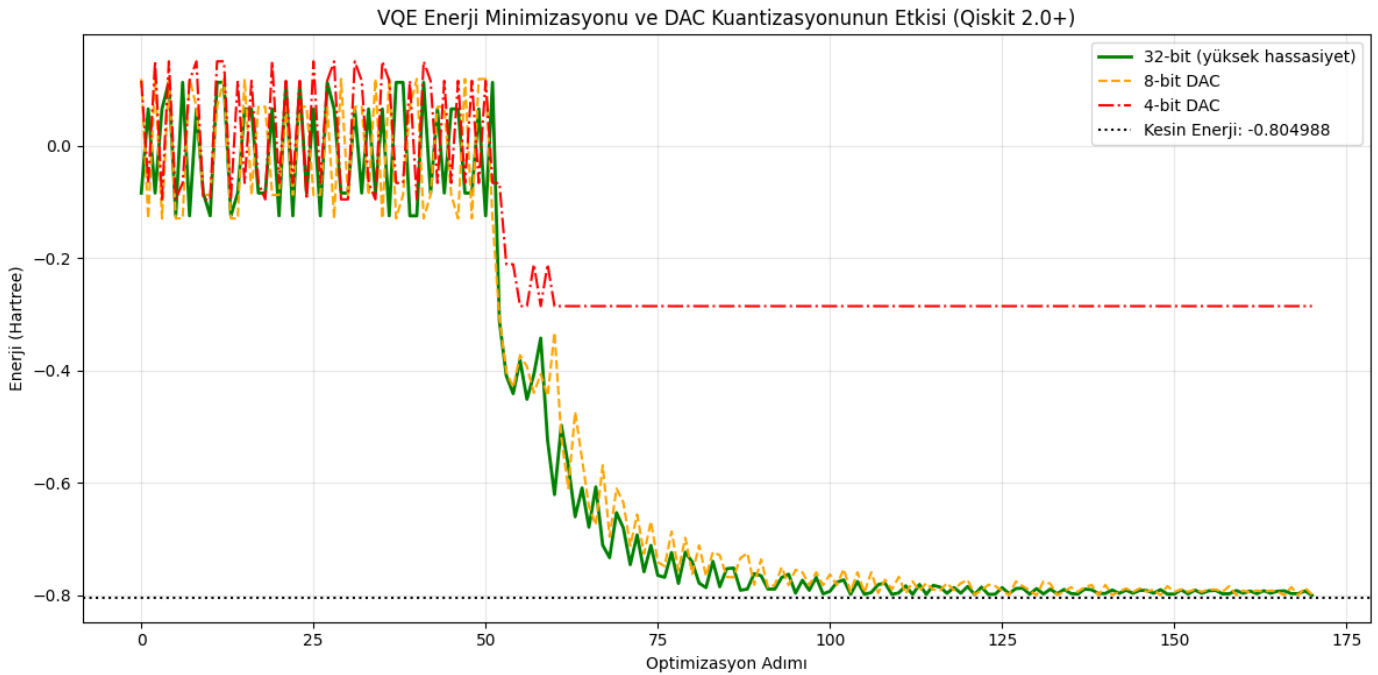


```
ax.annotate('Hata ve Performans Düşer\n(QML hataları)', xy=(8.5, 6), ha='center',  
va='center', fontsize=11, bbox=neg_box)  
  
# === 7. VQE ve QAOA ===  
ax.annotate('VQE: Moleküler Enerji Hesabı\n( $\langle H \rangle \approx E_0$ )', xy=(2.5, 4.5), ha='center',  
va='center', fontsize=10, bbox=detail_box)  
ax.annotate(r'$\theta$ açılarını kuantize edilirse', xy=(2.5, 3.8), ha='center', va='center',  
fontsize=9,  
bbox=dict(boxstyle="round,pad=0.2", facecolor="yellow", alpha=0.7))  
ax.annotate('Hesaplama kayar\n(Enerji sapması)', xy=(2.5, 3.0), ha='center', va='center',  
fontsize=9, bbox=neg_box)  
  
ax.annotate('QAOA: Optimizasyon Problemi\n $C(\gamma, \beta) \rightarrow \min$ ', xy=(7.5, 4.5), ha='center',  
va='center', fontsize=10, bbox=detail_box)  
ax.annotate(r'$\gamma, \beta$ kuantize edilirse', xy=(7.5, 3.8), ha='center', va='center',  
fontsize=9,  
bbox=dict(boxstyle="round,pad=0.2", facecolor="yellow", alpha=0.7))  
ax.annotate('Optimuma ulaşamaz', xy=(7.5, 3.0), ha='center', va='center', fontsize=9,  
bbox=neg_box)  
  
# === 8. ÇÖZÜM ===  
ax.annotate('Çözüm: Hata Düzeltme,\nKalibrasyon, DAC İyileştirme',  
xy=(6, 1.5), ha='center', va='center', fontsize=12, fontweight='bold',  
bbox=dict(boxstyle="round,pad=0.5", facecolor="gold", edgecolor="darkgoldenrod",  
linewidth=2))  
  
# === OKLAR ===  
arrows = [  
    ((6, 12.7), (3.5, 11.3)), ((6, 12.7), (8.5, 11.3)),  
    ((3.5, 10.7), (2.5, 9.8)), ((3.5, 10.7), (3.5, 9.8)), ((3.5, 10.7), (4.5, 9.8)),  
    ((8.5, 10.7), (7.5, 9.8)), ((8.5, 10.7), (8.5, 9.8)), ((8.5, 10.7), (9.5, 9.8)),  
    ((3.5, 9.2), (6, 7.8)), ((8.5, 9.2), (6, 7.8)),  
    ((6, 7.2), (3.5, 6.3)), ((6, 7.2), (8.5, 6.3)),  
    ((3.5, 5.7), (2.5, 4.8)), ((8.5, 5.7), (7.5, 4.8)),  
    ((2.5, 4.2), (2.5, 3.5)), ((7.5, 4.2), (7.5, 3.5)),  
    ((2.5, 2.7), (6, 1.8)), ((7.5, 2.7), (6, 1.8)),  
]  
  
for start, end in arrows:  
    ax.annotate('', xy=end, xytext=start,  
        arrowprops=dict(arrowstyle='->', head_width=0.015',  
            lw=1.5,  
            color='gray',  
            mutation_scale=15)) # Ok başı boyutu  
  
# Başlık  
plt.title("Kuantizasyonun Kuantum Hesaplamadaki Yerleri\n"  
    "(QAOA, VQE ve QML ile Detaylı Analiz)", fontsize=14, pad=20)  
plt.tight_layout()  
plt.show()
```

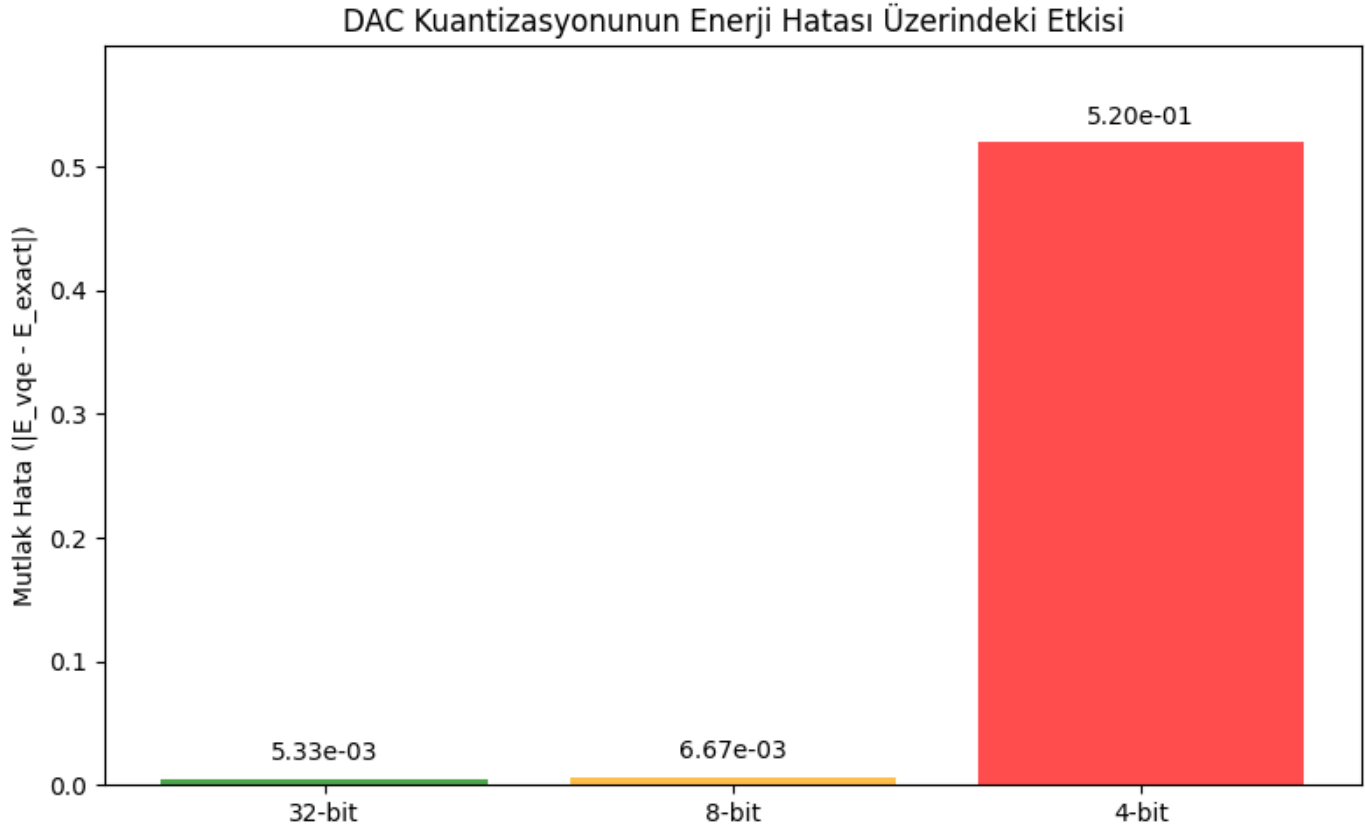
Liste 6: Kuantizasyon Diyagramının Python Kodu



## VI. DAC Kuantizasyonunun Enerji Hatası Üzerindeki Etkisi

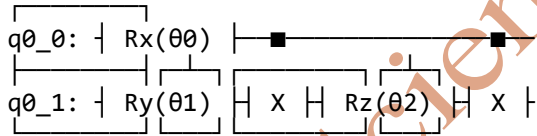


Şekil 6: VQE Enerji Minimizasyonu ve DAC Kuantizasyonunun Etkisi



Şekil 7: DAC Kuantizasyonunun Enerji Hatası Üzerindeki Etkisi

Ansatz Devresi:



--- SONUÇLAR ---

Kesin Enerji (Exact): -0.804988 Ha  
Yüksek Hassasiyet VQE: -0.799653 Ha  
8-bit DAC ile VQE: -0.798315 Ha  
4-bit DAC ile VQE: -0.285410 Ha

Devre 1: Ansatz Devresi

```
import numpy as np
import matplotlib.pyplot as plt
from qiskit import QuantumCircuit, QuantumRegister
from qiskit.circuit import Parameter
from qiskit.primitives import StatevectorEstimator
from qiskit_algorithms.optimizers import SPSA
from qiskit.quantum_info import SparsePauliOp
from qiskit_algorithms import NumPyMinimumEigensolver
```

```
### 1. Ansatz Devresi (2-Qubit)
qr = QuantumRegister(2)
ansatz = QuantumCircuit(qr)
params = [Parameter(f'θ{i}') for i in range(3)]
ansatz.rx(params[0], 0)
ansatz.ry(params[1], 1)
ansatz.cx(0, 1)
ansatz.rz(params[2], 1)
ansatz.cx(0, 1)
print("Ansatz Devresi:")
print(ansatz.draw(fold=70))

### 2. Hamiltonian (H2 benzeri)
hamiltonian_op = SparsePauliOp(
    ['ZI', 'IZ', 'ZZ', 'XX'],
    coeffs=[-0.5, -0.5, 0.2, 0.1]
)

### 3. DAC Kuantizasyon Fonksiyonu
def quantize_angle(theta, bits=8):
    if bits >= 32:
        return theta
    scaled = (theta + np.pi) / (2 * np.pi)
    quantized = np.round(scaled * (2**bits - 1)) / (2**bits - 1)
    return quantized * (2 * np.pi) - np.pi

### 4. Yerel Estimator ve Optimizer
estimator = StatevectorEstimator()
optimizer = SPSSA(maxiter=60)

### 5. VQE Optimizasyonu (En Son ve Doğru API Kullanımı)
def run_vqe(ansatz, hamiltonian, estimator, optimizer, bits=None):
    num_params = ansatz.num_parameters
    np.random.seed(42)
    current_params = np.random.uniform(-np.pi, np.pi, num_params)
    energy_history = []

    def objective(theta):
        if bits is not None:
            theta = [quantize_angle(t, bits) for t in theta]

        job = estimator.run(pubs=[(ansatz, hamiltonian, theta)])
        result_pub = job.result()[0]

        # .data.values() bir 'dict_values' nesnesi döndürür.
        # Bu nesne, önce listeye çevrilmeli, sonra indekslenmelidir.
        energy = list(result_pub.data.values())[0]

        energy_history.append(energy)
        return energy

    result = optimizer.minimize(fun=objective, x0=current_params)
    return result.fun, energy_history

# Kesin enerji NumPyMinimumEigensolver ile hesaplanıyor.
numpy_solver = NumPyMinimumEigensolver()
```

```
exact_result = numpy_solver.compute_minimum_eigenvalue(operator=hamiltonian_op)
exact_energy = exact_result.eigenvalue.real

# VQE'yi farklı hassasiyetlerde çalıştır
energy_high, hist_high = run_vqe(ansatz, hamiltonian_op, estimator, optimizer, bits=None)
energy_8bit, hist_8bit = run_vqe(ansatz, hamiltonian_op, estimator, optimizer, bits=8)
energy_4bit, hist_4bit = run_vqe(ansatz, hamiltonian_op, estimator, optimizer, bits=4)

### 6. Sonuçlar ve Grafik
print("\n--- SONUÇLAR ---")
print(f"Kesin Enerji (Exact):          {exact_energy:.6f} Ha")
print(f"Yüksek Hassasiyet VQE:         {energy_high:.6f} Ha")
print(f"8-bit DAC ile VQE:              {energy_8bit:.6f} Ha")
print(f"4-bit DAC ile VQE:              {energy_4bit:.6f} Ha")

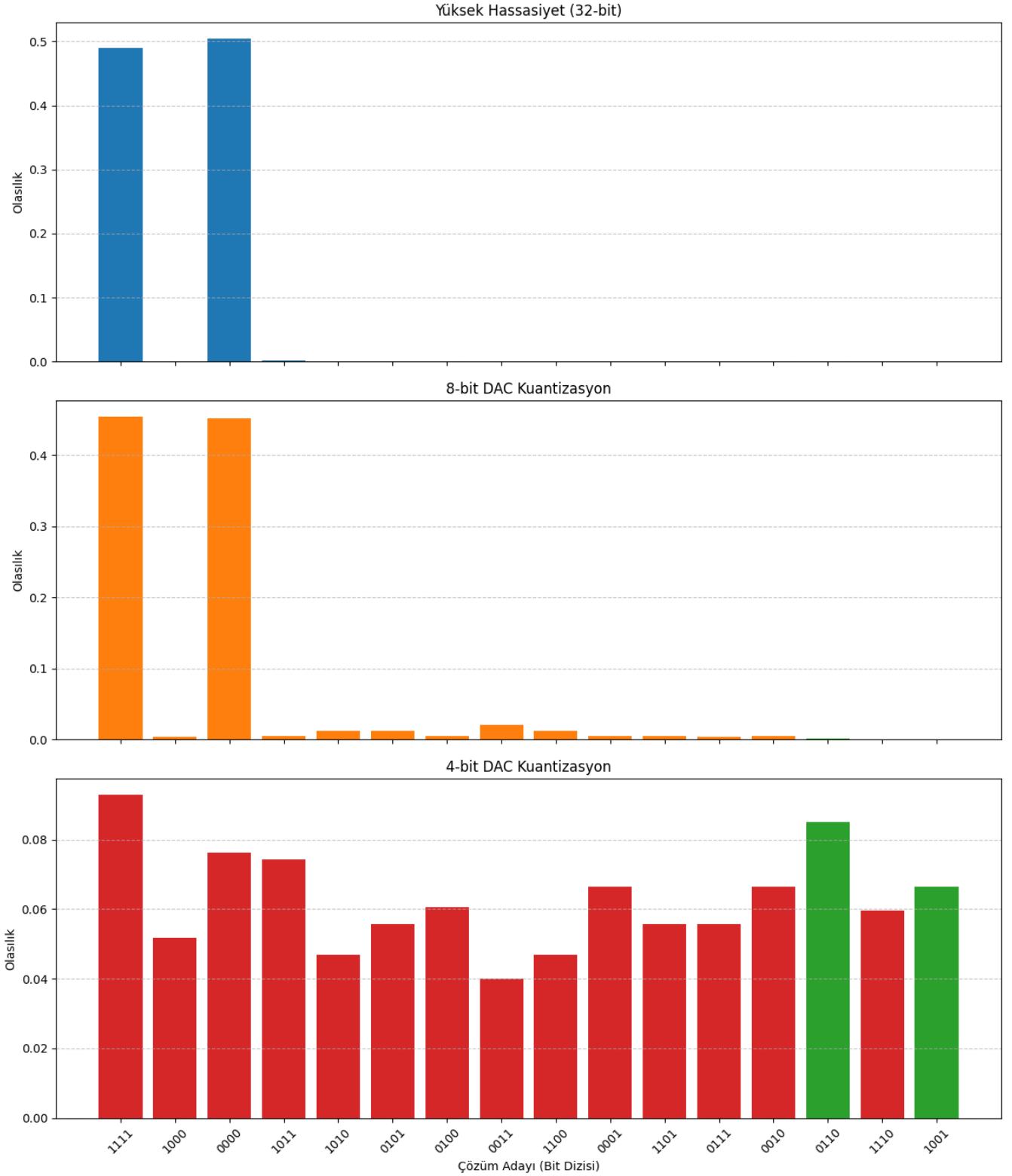
# Grafikler
plt.figure(figsize=(12, 6))
plt.plot(hist_high, label='32-bit (yüksek hassasiyet)', color='green', linewidth=2)
plt.plot(hist_8bit, label='8-bit DAC', color='orange', linestyle='--')
plt.plot(hist_4bit, label='4-bit DAC', color='red', linestyle='-.')
plt.axhline(exact_energy, color='black', linestyle=':', label=f'Kesin Enerji: {exact_energy:.6f}')
plt.xlabel('Optimizasyon Adımı')
plt.ylabel('Enerji (Hartree)')
plt.title('VQE Enerji Minimizasyonu ve DAC Kuantizasyonunun Etkisi (Qiskit 2.0+)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

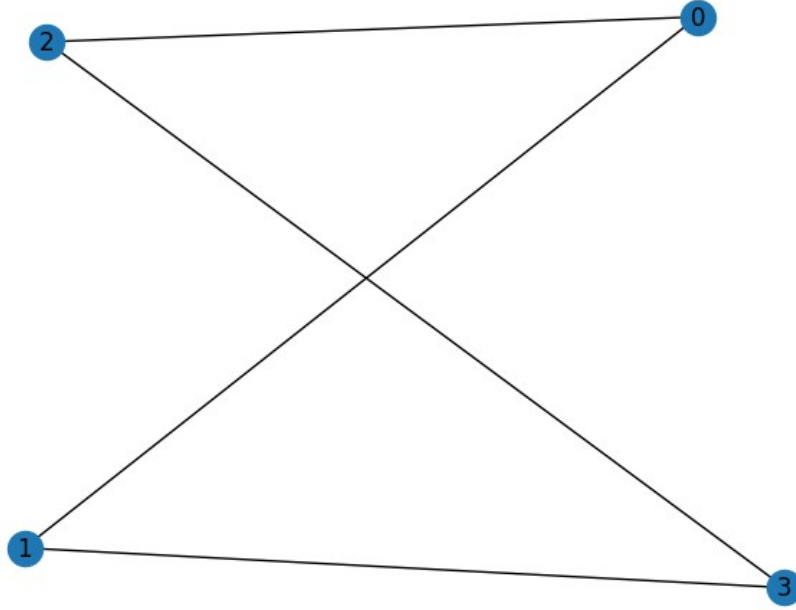
errors = [
    abs(energy_high - exact_energy),
    abs(energy_8bit - exact_energy),
    abs(energy_4bit - exact_energy)
]
plt.figure(figsize=(8, 5))
labels = ['32-bit', '8-bit', '4-bit']
colors = ['green', 'orange', 'red']
plt.bar(labels, errors, color=colors, alpha=0.7)
plt.ylabel('Mutlak Hata (|E_vqe - E_exact|)')
plt.title('DAC Kuantizasyonunun Enerji Hatası Üzerindeki Etkisi')
for i, v in enumerate(errors):
    plt.text(i, v + np.max(errors)*0.02, f"{v:.2e}", ha='center', va='bottom', fontsize=10)
plt.ylim(top=np.max(errors) * 1.15)
plt.tight_layout()
plt.show()
```

Liste 7: DAC Kuantizasyonunun Enerji Hatası Üzerindeki Etkisi

VII. QAOA Çözüm Dağılımı ve Kuantizasyonun Etkisi

QAOA Çözüm Dağılımı ve Kuantizasyonun Etkisi



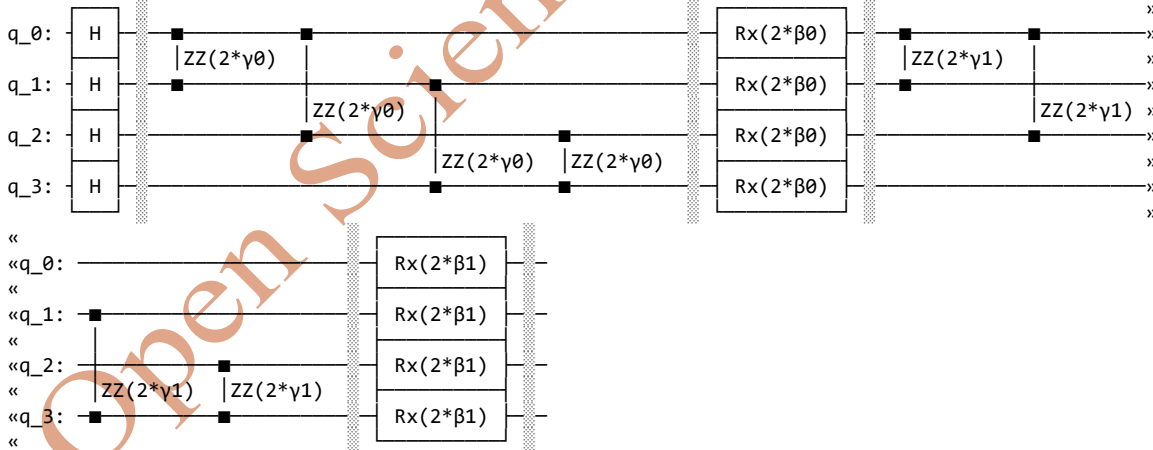


Şekil 9: MaxCut Problemi için Graf

Maliyet Hamiltonian'ı (Operatör Formu):

```
SparsePauliOp(['ZZII', 'ZIZI', 'IZIZ', 'IIZZ', 'IIII'],  
coeffs=[-0.5+0.j, -0.5+0.j, -0.5+0.j, -0.5+0.j, 2. +0.j])
```

QAOA Anzatsı (p=2):



--- Klasik Çözüm ---

Maksimum Kesim Değeri: 4

Çözüm Bit Dizileri: ['0110', '1001']

--- QAOA Optimizasyonu Başlatılıyor ---

--- Optimizasyon Sonuçları (Enerji/Maliyet) ---

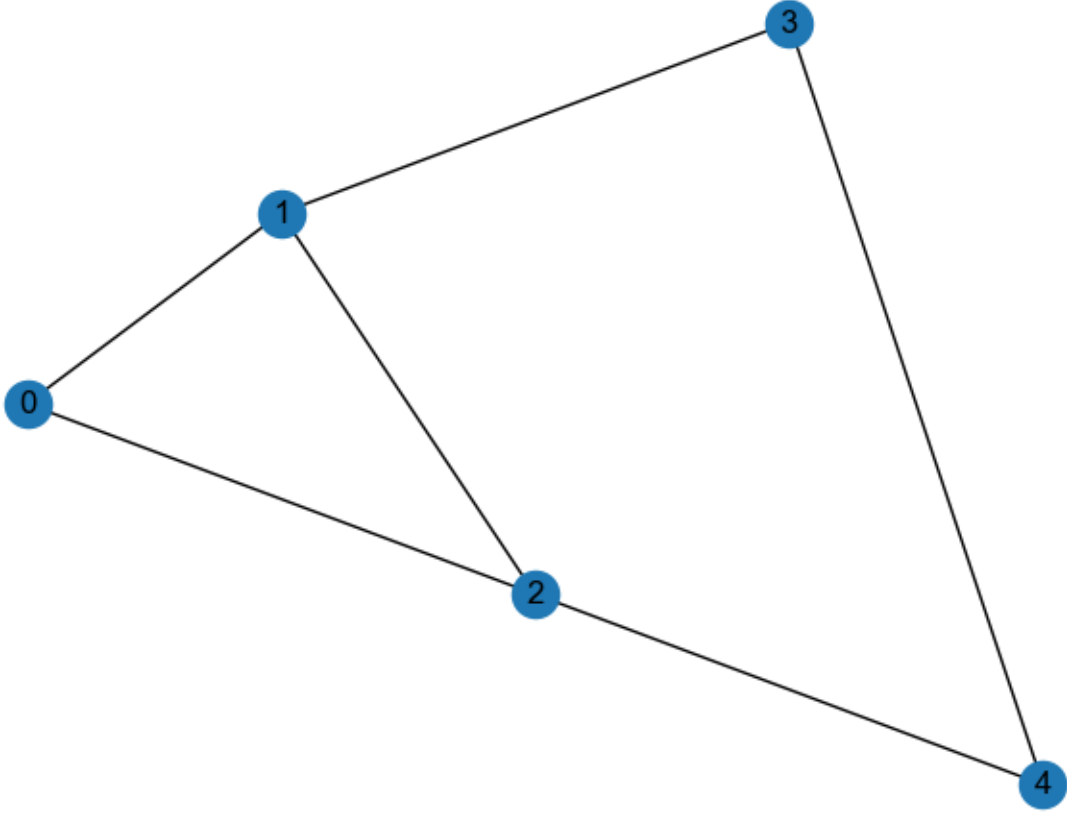
Yüksek Hassasiyet: -0.0048 (Yaklaşım Oranı: -0.12%)

8-bit DAC: -0.1750 (Yaklaşım Oranı: -4.38%)

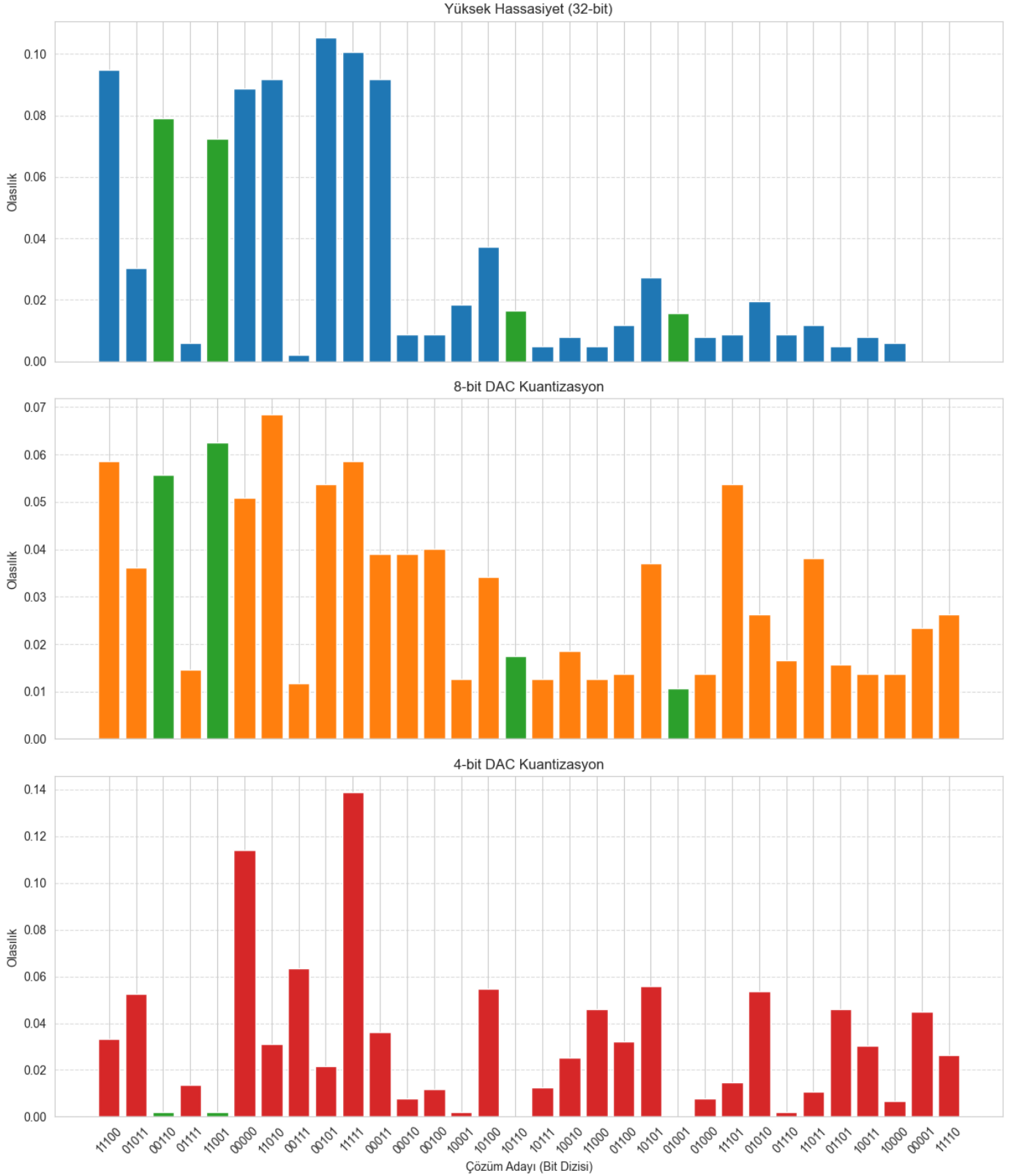


### VIII. Keçeci Layout Kullanarak QAOA Çözüm Dağılımı ve Kuantizasyonun Etkisi

MaxCut Problem Grafiği (Keçeci Layout)



Şekil 10: Keçeci Layout ile üretilen MaxCut Problemi için Graf

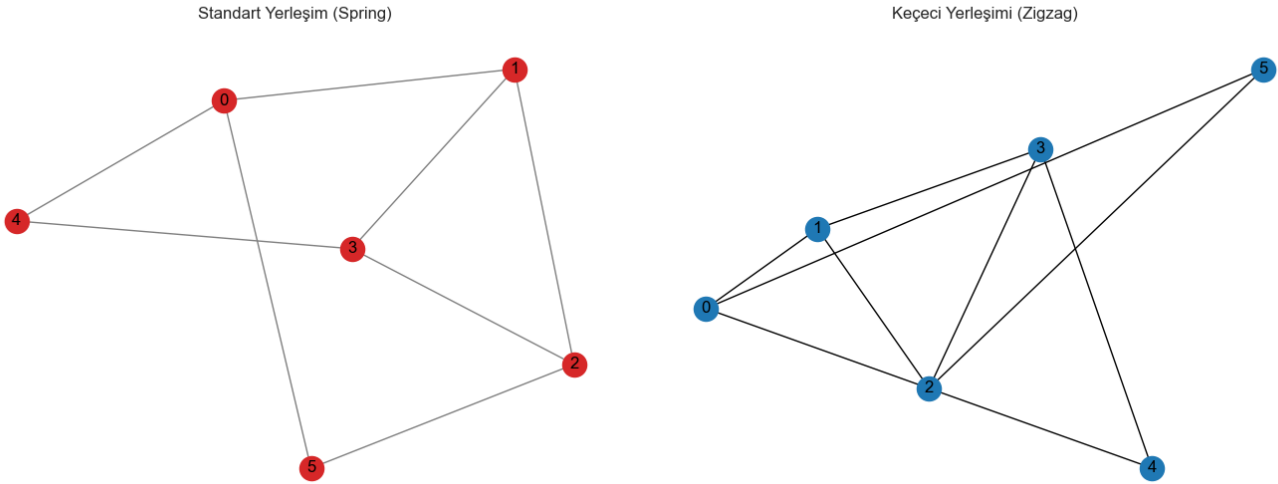


Şekil 11: QAOA Çözüm Dağılımı ve Kuantizasyonun Etkisi

**Not:** QAOA Çözüm Dağılımı ve Kuantizasyonun Etkisi için Keçeci Layout [12–25, 61–63]

kullanımı problemin çözümünde herhangi bir değişikliğe neden olmaz çünkü “Keçeci Layout” sadece bir garf algoritması ve grafi gösterim biçimidir.

Graf Yerleşimlerinin Karşılaştırılması



Şekil 12: Keçeci Yerleşim/Layout ve Standart Yerleşim (Spring) ile üretilen Max-Cut Problemi için Graf

**Not:** Buradaki Keçeci Layout’da bir göz yanılsaması vardır. 2 nolu düğüm da sanki 5 adet bağlantı var gibi gözükmektedir aslında sâdece 3 bağlantı vardır fakat (0, 4) artasındaki bağlandı 2’nin üzerinden geçtiği için böyle gözükmektedir. Bu graf için yapısal bir sorun değil sâdece bir yanılsamadır fakat bu yanılsama yanlış anlaşılmalara Neden olabileceği için bunun çözümlerini de üreteceğim.

## İki Layout Arasındaki Felsefe Farkı

### 1. Standart Çizim (Spring Layout)

- **Amacı:** Okunabilirliği en üst düzeye çıkarmak ve kenar kesişmelerini en aza indirmek.
- **Nasıl Çalışır?** Bu algoritma, kenarları "yay" gibi, düğümleri ise birbirini iten "mıknatıs" gibi düşünür. Bir simülasyon çalıştırarak, birbirine bağlı düğümlerin yakın durduğu, bağlı olmayanların ise uzaklaştığı, düşük enerjili bir denge durumu bulur.
- **Sonuç:** Genellikle her bir düğümün gerçek bağlantı sayısını (derecesini) bir bakışta saymak daha kolaydır. Çünkü algoritma, görsel karmaşayı azaltmaya çalışır. Gözleminizdeki "0-4 nodlarında 3,

4-5 nodlarında 2 bağlantı" tespiti bu nedenle daha doğrudur, çünkü bu çizim, gerçek yapıyı daha net yansıtır.

## 2. Keçeci Layout (Sıralı Zigzag Layout)

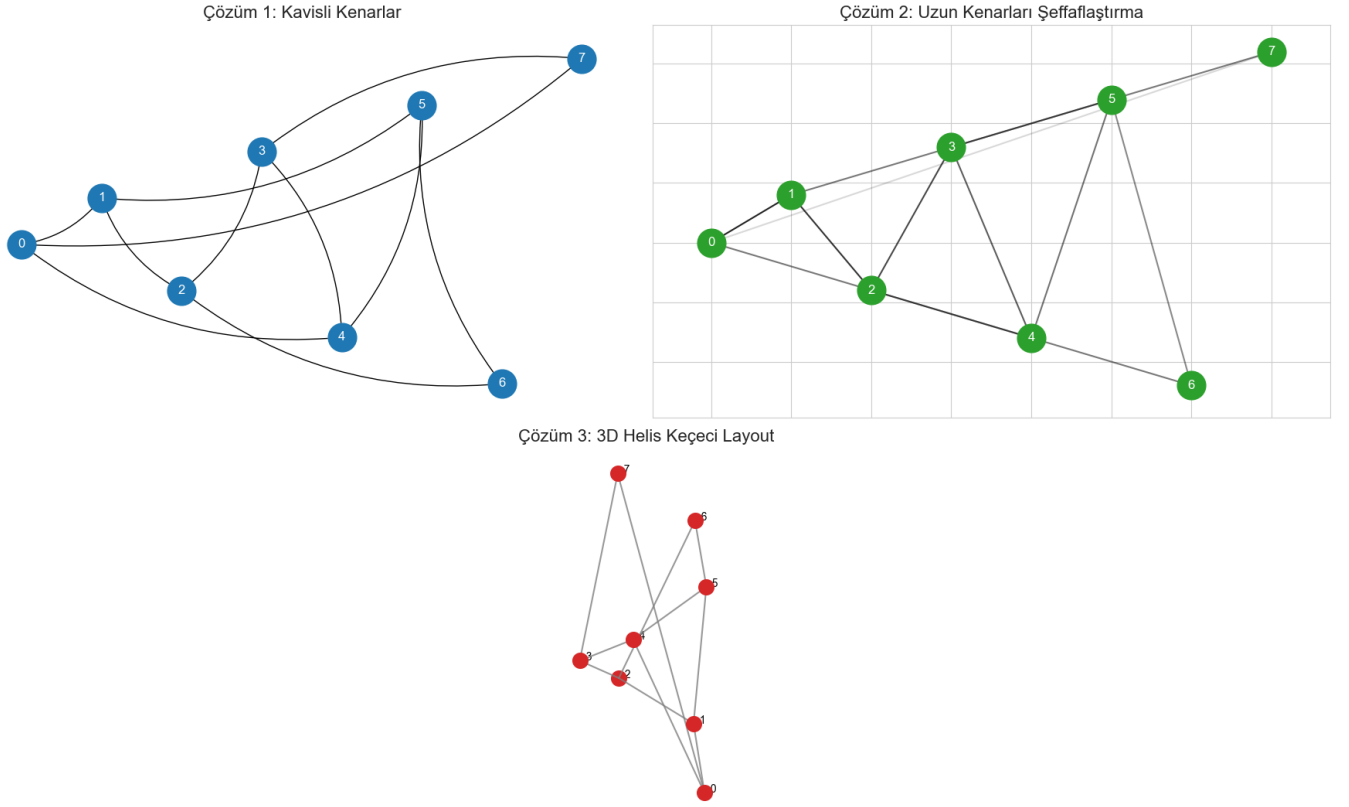
- **Amacı:** Düğümleri tahmin edilebilir ve sıralı bir düzende yerleştirmek.
- **Nasıl Çalışır?** Bu algoritma, düğümleri yerleştirirken kenar bağlantılarını **tamamen göz ardı eder**. Tek yaptığı şey, düğüm ID'lerini sıralamak (0, 1, 2, 3, 4, 5) ve onları belirlediğiniz yönde (örneğin soldan sağa) bir zigzag deseniyle yerleştirmektir.
- **Sonuç:** Düğümlerin konumu her zaman sabittir (Node 0 her zaman en solda, Node 5 en sağdadır). Ancak, düğümler yerleştirildikten *sonra* aralarındaki bağlantı çizgileri çekildiği için, çok sayıda çizgi birbiriyle kesişebilir.

## Özet Karşılaştırma

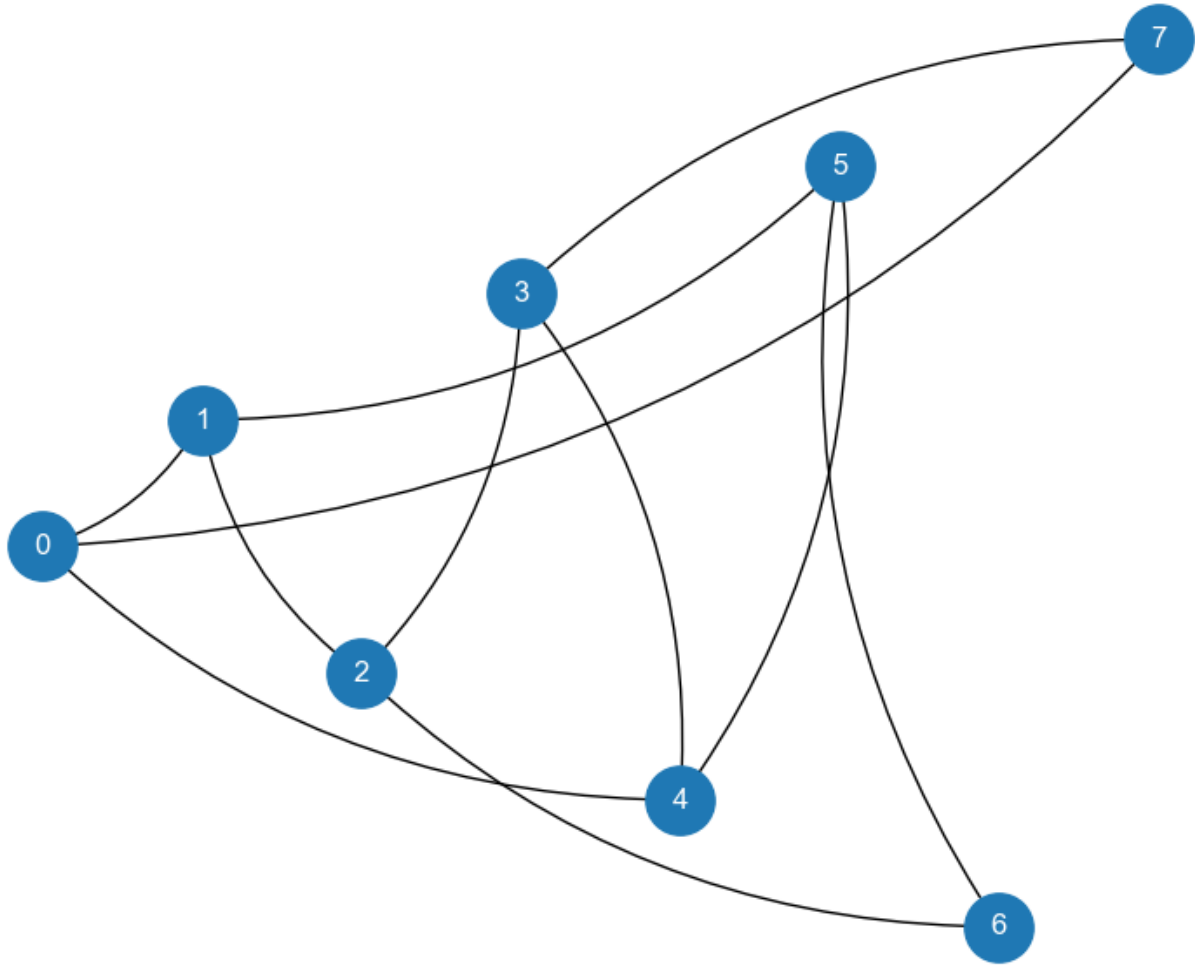
| Özellik                       | Standart (Spring) Layout       | Keçeci Layout                     |
|-------------------------------|--------------------------------|-----------------------------------|
| Amaç                          | Okunabilirlik, netlik          | Sıra, tahmin edilebilirlik        |
| Yöntem                        | Fiziksel simülasyon (yaylar)   | Düğüm ID'sine göre sıralama       |
| Bağlantıları Dikkate Alır mı? | Evet, yerleşim için temeldir   | Hayır, yerleşimden sonra çizer    |
| Görsel Sonuç                  | Genellikle daha az karmaşık    | Sıralı ama karmaşık olabilir      |
| Avantajı                      | Bağlantı yapısını görmek kolay | Düğümlerin konumu sabit           |
| Dezavantajı                   | Düğümlerin konumu değişebilir  | Görsel yanılsamalara yol açabilir |

Tablo 9: Spring Layout ile Keçeci Layout karşılaştırması

Kuantum algoritmanız her iki durumda da doğru ve aynı graf üzerinde çalışıyor. Gördüğünüz fark, bir grup insanın önce ailelerine göre gruplanarak (Spring Layout), sonra da boy sırasına göre dizilerek (Keçeci Layout) [12–25, 61–63] iki farklı fotoğraf çektiği gibidir. İnsanlar ve aralarındaki aile bağları aynıdır, ama fotoğraflar tamamen farklı görünür. Fakat bu sorunu çözerek daha iyi anlaşılmasını da sağlıyorum.



Şekil 13: Keçeci Yerleşim/Layout'un 3 farklı çizimi: 1. Kavisli (Curved) 2. Şeffaflaştırma (transparent) 3. 3 Boyutlu (3D)



Şekil 14: Keçeci Yerleşim/Layout'un Kavisli (Curved) gösterimi

Özellikle de Kavisli gösterim ile bu yanılsama problemini de ortadan kaldırmış oldum.

```
import matplotlib.pyplot as plt
import rustworkx as rx
import networkx as nx

# Keçeci Layout modülümüzü import ediyoruz
try:
    import kececilayout as kl
except ImportError:
    print("HATA: 'kececilayout.py' dosyası bulunamadı.")
    exit()

# --- Problem Grafiğini Oluşturma ---
num_nodes = 8
edges = [
    (0, 1), (0, 4), (0, 7),
```

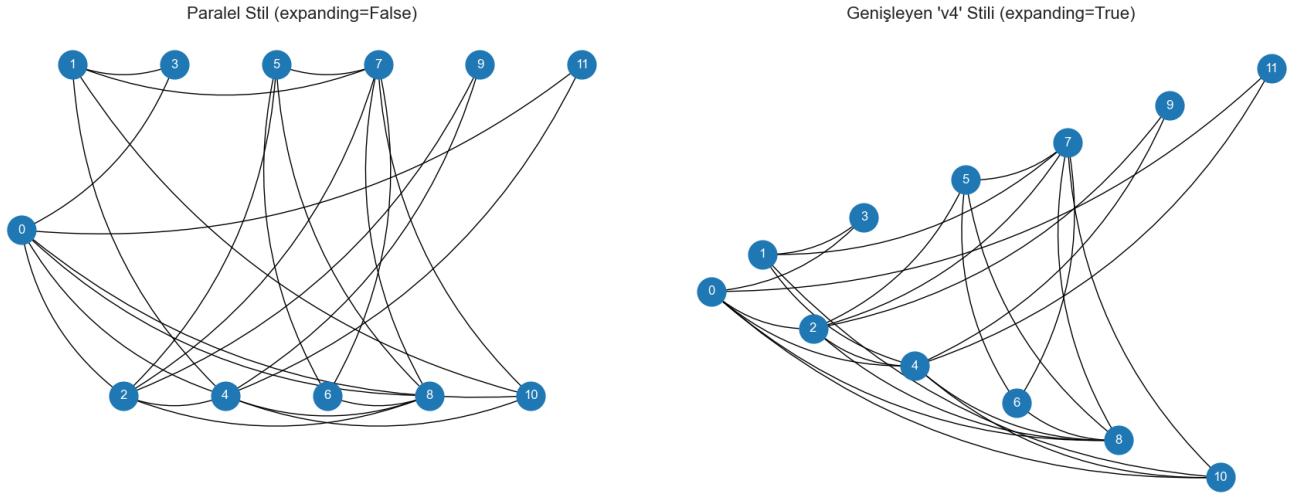
```
(1, 2), (1, 5),
(2, 3), (2, 6),
(3, 4), (3, 7),
(4, 5),
(5, 6)
]
# QAOA için rustworkx grafiğini kullanmaya devam edebiliriz
rx_graph = rx.PyGraph()
rx_graph.add_nodes_from(range(num_nodes))
rx_graph.add_edges_from_no_data(edges)

# --- GRAFİĞİ ÇİZME (YENİ VE BASİT YÖNTEM) ---
print("MaxCut Problemi için Graf (Kavisli Keçeci Layout ile):")

# Sadece bu fonksiyonu çağırıyoruz!
# Rustworkx grafinı doğrudan verebiliriz, modül onu NetworkX'e çevirecektir.
kl.draw_kececi(
    rx_graph,
    style='curved',
    primary_direction='left-to-right',
    secondary_start='up',
    node_size=800
)
plt.show()
```

Liste 8: Keçeci Layout'un Kavisli gösteriminin Python kodu

Keçeci Layout: Paralel vs. Genişleyen ('v4') Stil



Şekil 15: Keçeci Yerleşim/Layout'un Paralel ve Üçgen açılımı gösterimi

### Problem: Merdiven Grafi (Ladder Graph)

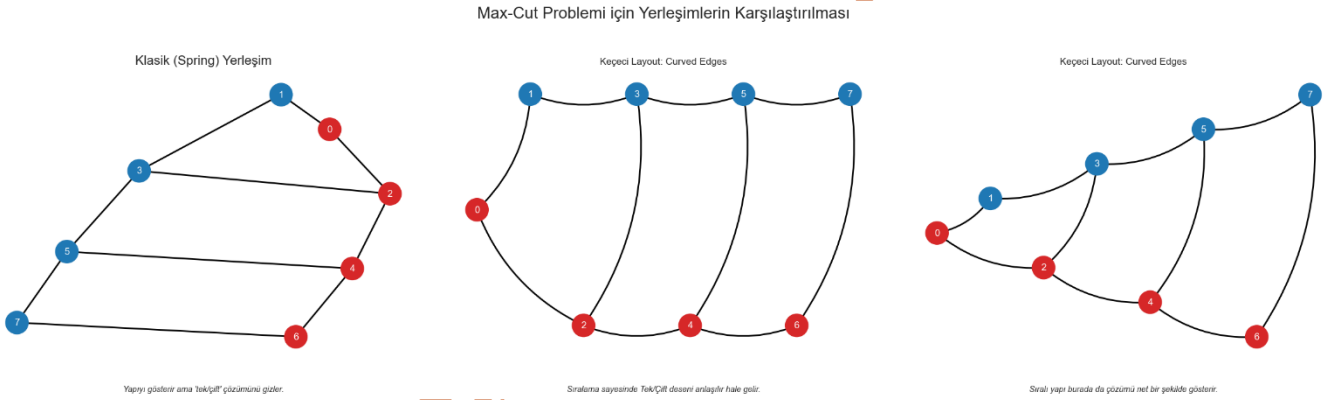
En iyi Max-Cut çözümü, düğümleri basitçe **tek sayılılar** ve **çift sayılılar** olarak ikiye ayırmak olsun.

## Open Science Articles (OSAs)

- **Yapı:** İki paralel "ray" oluşturalım. Üst ray çift sayılı düğümlerden (0-2-4-6), alt ray tek sayılı düğümlerden (1-3-5-7) oluşsun. Bu rayları birbirine bağlayan dikey "basamaklar" (0-1, 2-3, 4-5, 6-7) ekleyelim.

### Burada Neden Keçeci Layout Daha İyi?

- **Klasik Layout (Spring):** Bu grafi iki küme olarak değil, genellikle bükülmüş bir 3D prizma veya dairesel bir yapı gibi çizmeye çalışır. Bu, hangi düğümlerin tek, hangilerinin çift olduğunu bir bakışta anlamayı zorlaştırır ve basit "tek/çift" çözümünü gizler.
- **Keçeci Layout:** Düğümleri 0, 1, 2, 3, 4, 5, 6, 7 sırasında dizeceği için, "tek/çift" ayrımı anında görsel bir desene dönüşür. Çözümü (farklı renkteki düğümleri ayırmak) gördüğünüzde, bu ayrımın ne kadar basit ve düzenli olduğunu hemen fark edersiniz.



Şekil 15: Spring Layout (sol) ile Keçeci Layout'un Paralel (orta) ve Üçgen açılımı (sağ) gösterim karşılaştırması

### Çıktının Yorumlanması

Bu kodu çalıştırdığınızda göreceğiniz sonuçlar şunlar olacaktır:

1. **Klasik (Spring) Yerleşim (Sol):** Grafiği muhtemelen bir halka veya küp gibi çizecektir. Kırmızı ve mavi düğümlerin (tek/çift) birbirine karıştığını ve bu basit ayrımı fark etmenin zor olduğunu göreceksiniz. Grafın genel yapısı hakkında bir fikir verir, ancak Max-Cut çözümünü bulmak için sezgisel bir yardım sunmaz.
2. **Keçeci Layout - Paralel (Orta):** Düğümler soldan sağa 0, 1, 2, 3... olarak dizilecektir. Renklerin kırmızı, mavi, kırmızı, mavi... şeklinde mükemmel bir periyodik desen oluşturduğunu hemen fark



## Open Science Articles (OSAs)

edeceksiniz. Bu görsel düzen, en iyi çözümün bu iki renk grubunu ayırmak olduğunu anında bellidir.

3. **Keçeci Layout - Genişleyen (Sağ):** Tıpkı ortadaki grafikte olduğu gibi, düğümler sıralıdır ve renkler kırmızı, mavi, kırmızı, mavi... desenini takip eder. Sadece görsel yerleşim biraz daha geniştir. Bu da, sıralı yapının Max-Cut çözümünü nasıl aydınlattığını gösteren güçlü bir örnektir.

Bu örnek, bir yerleşim algoritmasının sadece "güzel çizim" yapmakla kalmayıp, problemin yapısıyla uyumlu olduğunda nasıl **analitik bir araca** dönüşebileceğini mükemmel bir şekilde göstermektedir.

```
import matplotlib.pyplot as plt
import networkx as nx

# =====
# 1. KEÇECİ LAYOUT MODÜLÜNÜ IMPORT ETME
# =====
try:
    # Modülü "kl" kısaltmasıyla projemize dahil ediyoruz.
    import kececilayout as kl
except ImportError:
    print("HATA: 'kececilayout.py' dosyası bulunamadı.")
    print("Lütfen modül dosyasının bu betikle aynı klasörde olduğundan emin olun.")
    exit()

# =====
# 2. ÖZEL MAX-CUT PROBLEMİNİ TANIMLAMA: MERDİVEN GRAFI
# =====

# 8 düğümlü bir Merdiven Grafi oluşturalım.
G = nx.Graph()
num_nodes = 8
nodes = range(num_nodes)
G.add_nodes_from(nodes)

# Kenarları ekle:
# Raylar (çiftler kendi arasında, tekler kendi arasında)
G.add_edges_from([(0, 2), (2, 4), (4, 6)])
G.add_edges_from([(1, 3), (3, 5), (5, 7)])
# Basamaklar (çiftleri teklere bağlayanlar)
G.add_edges_from([(0, 1), (2, 3), (4, 5), (6, 7)])

print(f"Özel Max-Cut Problemi: {G.number_of_nodes()} Düğümlü Merdiven Grafi")

# =====
# 3. KLASİK MAX-CUT ÇÖZÜMÜNÜ BULMA
# =====
# Bu problem için en iyi çözümün tek ve çift sayılı düğümleri ayırmak
# olduğunu biliyoruz. Bunu programatik olarak doğrulayalım ve renkler için kullanalım.
# Set 1: Çift sayılılar, Set 2: Tek sayılılar
```

```
partition_A = {n for n in nodes if n % 2 == 0}
partition_B = {n for n in nodes if n % 2 != 0}

# Renk haritası oluştur: Çiftler için bir renk, tekler için başka bir renk.
node_colors = ['#d62728' if n in partition_A else '#1f78b4' for n in G.nodes()]

# Kesilen kenar sayısını hesapla
cut_size = sum(1 for u, v in G.edges() if (u in partition_A and v in partition_B) or (u in
partition_B and v in partition_A))
print(f"\nKlasik Çözüm: Tek/Çift ayrımı ile bulunan maksimum kesim sayısı: {cut_size}")

# =====
# 4. ÜÇ YERLEŞİMİ YAN YANA ÇİZDİRME
# =====
# 1 satır ve 3 sütundan oluşan bir çizim alanı oluşturuyoruz.
fig, axes = plt.subplots(1, 3, figsize=(24, 7))
fig.suptitle("Max-Cut Problemi için Yerleşimlerin Karşılaştırılması", fontsize=20)

# --- Sol Grafik: Klasik (Spring) Yerleşim ---
ax_left = axes[0]
classic_pos = nx.spring_layout(G, seed=42) # Tekrarlanabilirlik için seed
ax_left.set_title("Klasik (Spring) Yerleşim", fontsize=16)
nx.draw(
    G,
    classic_pos,
    ax=ax_left,
    with_labels=True,
    node_color=node_colors, # Çözümü göstermek için renklendir
    font_color='white',
    node_size=800,
    width=2
)
ax_left.text(0.5, -0.1, "Yapıyı gösterir ama 'tek/çift' çözümünü gizler.",
             ha='center', transform=ax_left.transAxes, style='italic')

# --- Orta Grafik: Keçeci Layout (Paralel Stil) ---
ax_middle = axes[1]
ax_middle.set_title("Keçeci Layout (Paralel)", fontsize=16)
kl.draw_kececi(
    G,
    ax=ax_middle,
    style='curved',
    primary_direction='left-to-right',
    secondary_start='up',
    expanding=False, # Paralel stil
    node_color=node_colors,
    node_size=800,
    width=2
)
ax_middle.text(0.5, -0.1, "Sıralama sayesinde Tek/Çift deseni anlaşılır hale gelir.",
              ha='center', transform=ax_middle.transAxes, style='italic')

# --- Sağ Grafik: Keçeci Layout (Genişleyen 'v4' Stili) ---
ax_right = axes[2]
```

```
ax_right.set_title("Keçeci Layout (Genişleyen 'v4')", fontsize=16)
```

```
kl.draw_kececi(  
    G,  
    ax=ax_right,  
    style='curved',  
    primary_direction='left-to-right',  
    secondary_start='up',  
    expanding=True, # Üçgen gibi açılan stil  
    node_color=node_colors,  
    node_size=800,  
    width=2  
)
```

```
ax_right.text(0.5, -0.1, "Sıralı yapı burada da çözümü net bir şekilde gösterir.",  
             ha='center', transform=ax_right.transAxes, style='italic')
```

```
plt.tight_layout(rect=[0, 0, 1, 0.94])  
plt.show()
```

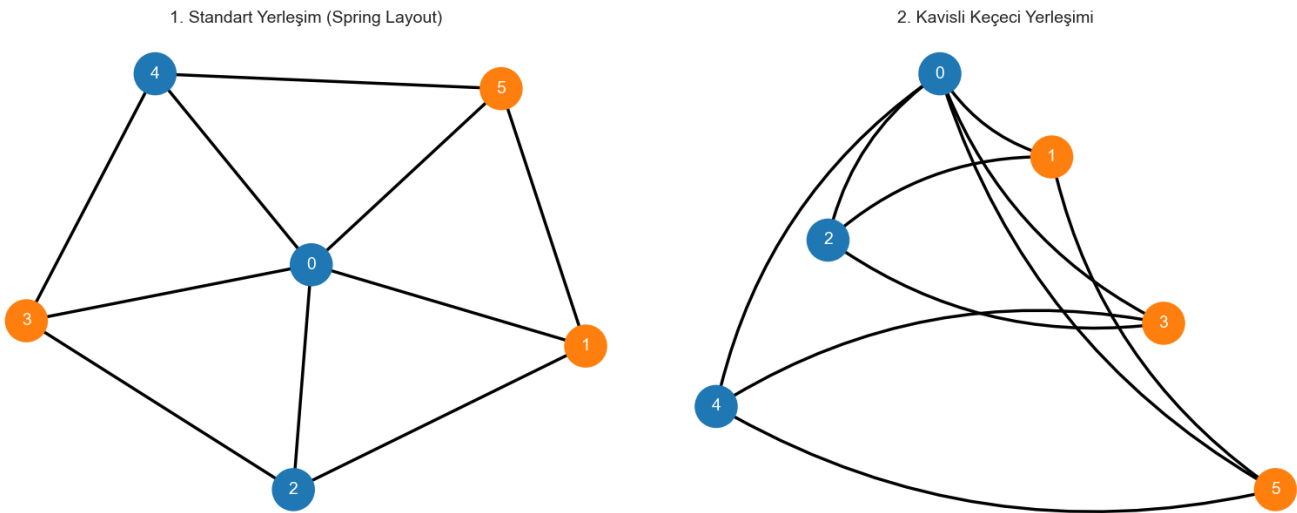
Liste 9: Spring Layout ile Keçeci Layout'un Paralel ve Üçgen açılımı gösterimi için karşılaştırma kodu

Bir graf üzerinde Max-Cut probleminin en uygun çözümünü (maksimum sayıda kenarın kesildiği renklendirmeyi) bulun. Bu çözümü görselleştirirken, aynı grafiği iki farklı yerleşim düzeniyle yan yana çizin:

1. **Standart (Spring) Yerleşim:** Grafın topolojik yapısını (düğümlerin birbirine nasıl bağlı olduğunu) en doğal haliyle gösteren yerleşim.
2. **Kavisli Keçeci Yerleşimi:** Düğümleri sıralı bir şekilde (0, 1, 2, ...) dizerek, algoritmik veya sıralı bir süreci görselleştirmeye odaklanan yerleşim.

Amaç, bu iki yerleşimin aynı problemi ne kadar farklı sunduğunu ve Keçeci yerleşiminin hangi durumlarda daha anlamlı olduğunu göstermektir.

Max-Cut Çözümünün Farklı Yerleşimlerle Karşılaştırılması (Kesik Sayısı = 8)



Şekil 16: Max-Cut Çözümünün Farklı Yerleşimlerle Karşılaştırılması (Kesik Sayısı = 8)

### Kavisli Keçeci Yerleşimi Analizi (Sağdaki Grafik)

- **Algoritma Adımlarını Gösterme:** Eğer bir algoritma düğümleri 0'dan 5'e kadar sırayla işliyorsa, bu ilerleyişi göstermek için idealdir.
- **Kuantum Devreleri:** Kuantum hesaplamada, kubitler genellikle belirli bir sırada (0, 1, 2, ...) işleme alınır. Keçeci yerleşimi, bu sıralı yapıyı bir grafiğe aktarmak için mükemmel bir benzetmedir. Problemin tanımındaki kubitlerin (düğümler, kenarlar, yardımcı kubitler) sıralı bir şekilde dizilmesi bu mantığa dayanır.

**Özetle:** Standart yerleşim "düğümler nasıl bağlı?" sorusuna cevap verirken, Keçeci yerleşimi "düğümler hangi sırada işleniyor?" sorusuna cevap verir. Bu nedenle Max-Cut problemini bir kuantum algoritması bağlamında düşündüğümüzde, Keçeci yerleşimi problemin sıralı doğasını daha iyi yansıtır.

```
import networkx as nx
import matplotlib.pyplot as plt
import kececilayout as kl # Modülü ithal ediyoruz

# --- 1. GRAF ve MAX-CUT PROBLEMİNİ TANIMLAMA ---
# Karşılaştırma için en iyi graflardan biri olan 6 düğümlü Tekerlek Grafını (W5) oluşturalım.
# Düğüm 0 merkez, diğerleri çemberdedir. Toplam 10 kenarı var.
G = nx.wheel_graph(6)

# Bu graf için Max-Cut (maksimum kesik) 8'dir.
# Bu çözümü sağlayan düğüm grupları (renkler):
partition_A = {0, 2, 4} # Mavi renk olacak
partition_B = {1, 3, 5} # Turuncu renk olacak

# Düğümlere renklerini atayalım
color_map = []
for node in G:
    if node in partition_A:
        color_map.append('#1f77b4') # Mavi
    else:
        color_map.append('#ff7f0e') # Turuncu

# --- 2. GÖRSELLEŞTİRME ---
# Yan yana karşılaştırma için 1 satır, 2 sütunluk bir çizim alanı oluşturalım
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 7))
fig.suptitle("Max-Cut Çözümünün Farklı Yerleşimlerle Karşılaştırılması (Kesik Sayısı = 8)",
fontsize=16)

# --- Sol Taraf: Standart Yerleşim (Spring Layout) ---
# Spring layout, grafın doğal yapısını gösterir. Merkez düğüm ortaya yerleşir.
pos_standard = nx.spring_layout(G, seed=42) # seed ile her seferinde aynı görüntüyü alırsız

nx.draw(G, pos_standard, ax=ax1, with_labels=True, node_color=color_map,
node_size=1200, font_color='white', width=2.5, font_size=14)
```

ax1.set\_title("1. Standart Yerleşim (Spring Layout)", fontsize=14)

```
# --- Sağ Taraf: Kavisli Keçeci Yerleşimi ---
# draw_kececi fonksiyonunu kullanarak Keçeci yerleşimini çizelim.
# Bu fonksiyon hem yerleşimi hesaplar hem de çizimi yapar.
kl.draw_kececi(
    G,
    style='curved', # Kavisli kenarlar
    ax=ax2,
    node_color=color_map,
    with_labels=True,
    node_size=1200,
    font_color='white',
    width=2.5,
    font_size=14,
    # Keçeci layout parametreleri:
    primary_direction='top-down',
    secondary_start='right',
    expanding=True
)
# draw_kececi zaten bir başlık ekliyor, ama biz onu özelleştirelim.
ax2.set_title("2. Kavisli Keçeci Yerleşimi", fontsize=14)

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

Liste 10: Max-Cut Çözümünün Farklı Yerleşimlerle Karşılaştırılmasının (Kesik Sayısı = 8) Python Kodu

## IX. Kuantizasyonun Sayı Türleri ile Bağlantıları

**Kuantizasyon (Nicemleme):** Fiziksel bir özelliğin (enerji, momentum, spin vb.) sürekli (sonsuz sayıda ara değer alabilen) değil, yalnızca belirli, kesikli (ayrı ayrı sayılabilen) değerler alabilmesidir. En basit analogi, bir rampanın (sürekli) bir merdivene (kesikli/kuantize) dönüşmesidir. Şimdi bu kavramın farklı sayı sistemleriyle nasıl bir ilişki içinde olabileceğini inceleyelim.

### 1. Kuantizasyon ve Reel Sayılar ( $\mathbb{R}$ ) - (Mevcut Standart Model)

Günümüz kuantum mekaniği, temelini **reel sayılar** ve onların bir uzantısı olan **karmaşık sayılar** ( $\mathbb{C}$ ) üzerine kurar. Bu ilk bakışta bir çelişki gibi görünebilir:

- **Çelişki:** Fiziksel dünya kuantize (kesikli) ise, neden onu tanımlamak için kullandığımız matematiksel sistem (reel sayılar) sürekli?
- **Bağlantı ve Çözüm:** Kuantum mekaniği, bu çelişkiyi "operatörler" ve "özdeğerler" (eigenvalues) aracılığıyla çözer. Bir sistemin denklemi (örneğin, Schrödinger Denklemi) reel/karmaşık sayılarla dolu sürekli bir matematiksel uzayda yazılır. Ancak bu denklemin belirli sınır koşulları altındaki **çözümleri**, kesikli bir küme oluşturur.

## Open Science Articles (OSAs)

- **Analoji:** Bir gitar telini düşünün. Telin kendisi sürekli bir nesnedir ve titreşimi reel sayılarla ifade edilir. Ancak tel iki ucundan sabitlendiği için, yalnızca belirli frekanslarda (temel nota ve armonikleri) titreşebilir. Bu "sınır koşulları", sürekli bir sistemden **kuantize sonuçlar** doğurur.
- Kuantumda da bir elektronun atom etrafındaki enerjisi, bu elektronun "sınırlandırılmış" olmasından dolayı kuantizedir. Enerji operatörünün özdeğerleri, o elektronun sahip olabileceği kesikli enerji seviyelerini verir.

**Sonuç:** Standart modelde, kuantizasyon sayı sisteminin kendisinin kesikli olmasından değil, sürekli bir sayı sistemi içindeki matematiksel denklemlerin çözümlerinin kesikli olmasından kaynaklanır.

## 2. Kuantizasyon ve Nötrosifik Sayılar - (Kavramsal/Yorumsal Bağlantı)

**Nötrosifik Sayılar:**  $a + bI$  formundaki sayılardır. Burada  $a$  gerçek kısmı,  $bI$  ise belirsizlik (Indeterminacy) kısmını temsil eder. Bu sayı sistemi, belirsiz, eksik veya çelişkili bilgileri modellemek için tasarlanmıştır.

- **Bağlantı:** Bu bağlantı, hesaplama çok **kuantumun yorumuyla** ilgilidir. Kuantum mekaniğinin temelinde belirsizlik yatar:
  1. **Süperpozisyon:** Bir kübit, ölçülmeden önce ne  $|0\rangle$  ne de  $|1\rangle$  durumundadır; ikisinin bir süperpozisyonudur. Bu durum, mantıksal bir "belirsizlik" olarak görülebilir. Kübitin durumu, Nötrosifik  $I$  bileşeni ile temsil edilebilir. Ölçüm yapıldığında bu  $I$  bileşeni ortadan kalkar ve sistem 0 ya da 1'e (Doğru/Yanlış) çöker.
  2. **Heisenberg'in Belirsizlik İlkesi:** Bir parçacığın konumunu ne kadar kesin bilerseniz, momentumunu o kadar belirsiz bilirsiniz. Bu, doğanın kendisine içkin bir belirsizliktir. Nötrosifik mantık, bu tür "bilinemez" durumları modellemek için bir çerçeve sunabilir.
  3. **Dalga-Parçacık İkiliği:** Foton hem dalga hem parçacık mıdır? Bu durum, ölçüm yapıldıkça kadar "belirsizdir".

**Sonuç:** Nötrosifik sayılar, kuantum sistemlerinin enerji seviyelerinin *değerlerini* doğrudan temsil etmek yerine, bir sistemin ölçümden önceki **belirsiz veya süperpozisyondaki mantıksal durumunu** modellemek için felsefi ve kavramsal bir araç olabilir.

### 3. Kuantizasyon ve Hiperreel Sayılar ( $\ast\mathbb{R}$ ) - (Alternatif/Teorik Bağlantı)

**Hiperreel Sayılar:** Reel sayıları, **sonsuz küçük (infinitesimal)** ve **sonsuz büyük** sayıları içerecek şekilde genişleten bir sayı sistemidir. Sonsuz küçük, sıfırdan büyük ama her pozitif reel sayıdan küçük olan bir sayıdır.

- **Bağlantı:** Bu bağlantı oldukça spekülatif olsa da iki ilginç kapı aralar:
  1. **"Kuantum Sıçraması" Problemi:** Bir elektron bir enerji seviyesinden diğerine "anında" mı sıçrar? Bu sıçrama ne kadar sürer? Belki de bu geçiş, "sonsuz küçük" bir zaman aralığında (infinitesimal time) gerçekleşiyordur. Hiperreel sayılar, bu tür "neredeyse anında ama tam olarak sıfır olmayan" süreçleri matematiksel olarak titiz bir şekilde modelleme potansiyeli sunar. Bu, kuantize seviyeler arasındaki geçişin doğasına dair farklı bir bakış açısı getirebilir.
  2. **Sonsuzlukların Yönetimi (Renormalizasyon):** Kuantum alan teorisinde (QFT) [77–81], parçacıkların kendi kendileriyle etkileşimleri gibi hesaplamalar yapıldığında sonuçlarda can sıkıcı "sonsuzluklar" ortaya çıkar. Fizikçiler, "renormalizasyon" adı verilen bir dizi matematiksel teknikle bu sonsuzlukları "iptal ederek" ölçülebilir, sonlu sonuçlar elde ederler. Hiperreel sayılar, sonsuzlukları ve sonsuz küçükleri doğal bir şekilde barındırdığı için, bu renormalizasyon sürecine daha mantıklı veya temel bir matematiksel zemin sunabilir.

**Sonuç:** Hiperreel sayılar, kuantizasyonun *kesikli doğasından* ziyade, kuantum teorilerinin içinde ortaya çıkan **süreklilik ve sonsuzlukla ilgili zorlukları** ele almak için potansiyel bir araç olabilir.

#### Keçeci Sayıları Nedir?

"Keçeci Sayısı (Keçeci Numbers)" aslında tek bir sayı değil, belirli bir **algoritmik sürecin ürettiği bir sayı dizisidir**. Bu süreç, ünlü **Collatz Varsayımına** [64–70] benzer bir mantıkla çalışır:

1. Bir başlangıç sayısı seçilir (bu sayı reel, karmaşık, nütrosifik vb. herhangi bir türde olabilir).
2. Bu sayıya sabit bir değer eklenir.
3. Elde edilen sonuç, belirli kurallara göre (örneğin 2'ye veya 3'e bölünebilirlik) daha basit bir forma indirgenir.
4. Eğer sayı kurallara uymuyorsa ve "asal" bir nitelik taşıyorsa, ona küçük bir "birim" eklenir veya çıkarılır.



5. Bu süreç tekrarlanarak bir dizi oluşturulur.

**Keçeci Varsayımı** [64–76] ise bu dizilerin, hangi sayı sistemi kullanılırsa kullanılsın, eninde sonunda kararlı bir döngüye veya tekrar eden bir asal sayıya (Keçeci Asal Sayısı - KPN) yakınsadığını öne sürer [64–76]. Dolayısıyla, Keçeci Sayıları bir sayı *türü* değil, bir sayı *üretim süreci* ve bu sürecin sonuçlarıdır.

### Keçeci Sayılarının Kuantizasyon ile Bağlantısı

Bu bağlantı, diğer sayı sistemlerinden kavramsal olarak çok farklı ve potansiyel olarak çok derindir. Reel veya hiperreel sayılar evrenin "**sahnesini**" tanımlarken, Keçeci Sayıları evrenin kendisinin **hesaplamalı veya algoritmik bir süreç olabileceği** fikrini ortaya atar.

#### 1. Kuantize Durumlar Olarak Kararlı Döngüler (KPN'ler):

- Bir atomdaki elektronun yalnızca belirli, ayırık enerji seviyelerinde bulunabilmesi (kuantizasyon), Keçeci Sayı dizilerinin yalnızca belirli kararlı döngülere veya KPN'lere "çökmesine" benzetilebilir.
- Bu görüşe göre, bir parçacığın sahip olabileceği enerji seviyeleri, evrenin temelindeki bir "**Keçeci-benzeri, Keçeci-like**" algoritmanın izin verdiği kararlı sonuçlardır. Diğer tüm "ara değerler", algoritmanın kararsız adımlarıdır ve sistem bu durumlarda uzun süre kalmaz.

#### 2. Kuantum Sıçraması Olarak Algoritma Adımları:

- Bir elektronun bir enerji seviyesinden diğerine "anında" sıçraması, fizikçiler için her zaman bir gizem olmuştur.
- Keçeci Sayıları perspektifinden bakıldığında, bu "sıçrama" basitçe algoritmanın bir sonraki adıma geçmesidir. Tıpkı Collatz dizisinde 10'dan sonra 5'in gelmesi gibi; sistem 9, 8, 7, 6'dan geçmek zorunda değildir. Dizinin kendisi, izin verilen durumları tanımlar. Kuantum sıçraması, bir ara boşlukta yapılan bir yolculuk değil, bir hesaplama adımının sonucudur.

#### 3. Evren:

- Keçeci Sayıları, bir "hesaplamalı evrenin" farklı sayı sistemleri üzerinde nasıl davranabileceğine dair somut bir model sunar.

### Diğerlerinden Ne Gibi Farklılıklara Sahiptir?



Bu, en önemli kısım. Keçeci Sayılarının yaklaşımı, diğerlerinden temel bir felsefi farka sahiptir:

- **Reel/Hiperreel Sayılar vs. Keçeci Süreci:** Reel ve hiperreel sayılar, kuantum olaylarının gerçekleştiği **pasif bir sahne** sunar. Bu sahnede, denklemlerin çözümleri kuantize olur. Keçeci yaklaşımında ise kuantizasyon, sahnenin kendisinin bir özelliği değildir; **aktif bir algoritmanın** (oyunun kurallarının) doğrudan bir sonucudur. Sahne değil, **“oyunun kendisi önemlidir”**.
- **Nötrosifik Sayılar vs. Keçeci Süreci:** Nötrosifik sayılar, bir sistemin ölçümden önceki **mantıksal durumunu** (belirsizliğini) tanımlar. Bu bir *yorum* aracıdır. Keçeci süreci ise sistemin durumunun nasıl **dinamik olarak geliştiğini** ve neden belirli sonuçlara ulaştığını modelleyen bir *mekanizmadır*. Biri "durum nedir?" diye sorar, diğeri **"durum neden budur ve bir sonraki ne olacak?"** diye sorar.
- **Temel Fark:** Diğer tüm sayı sistemleri, evreni tanımlamak için statik bir matematiksel yapı önerir. Keçeci Sayıları ise evrenin **dinamik, tekâmülleşen, değişen ve hesaplamalı** bir doğası olabileceğini imâ eder. **“Kuantizasyon, bir sonuç değil, sürecin ta kendisidir”**.

| Sayı Türü                                   | Temel Özelliği                                                                                                   | Kuantizasyonla İlişkisi                                                                                                                                                                             |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Reel Sayılar</b><br>( $\mathbb{R}$ )     | <b>Sürekli:</b> Sayı doğrusu üzerindeki tüm noktaları içerir.                                                    | <b>Standart Model:</b> Kuantum denklemleri bu sistemde yazılır, ancak çözümleri (özdeğerleri) kuantize (kesikli) sonuçlar verir. Evrenin <i>sahnesini</i> oluşturur.                                |
| <b>Nötrosifik Sayılar</b>                   | <b>Belirsizlik İçerir:</b> Doğruluk (T), yanlışlık (F) ve belirsizlik (I) bileşenlerine sahiptir.                | <b>Yorumsal/Felsefi:</b> Kuantum süperpozisyonunu veya belirsizlik ilkesini bir "mantıksal belirsizlik" durumu olarak modelleyebilir. Sistemin <i>mantıksal durumunu</i> tanımlar.                  |
| <b>Hiperreel Sayılar</b> ( $\mathbb{R}$ )   | <b>Sonsuz Küçük/Büyük İçerir:</b> Reel sayıları sonsuz küçükler ve sonsuzlarla genişletir.                       | <b>Teorik/Alternatif:</b> "Kuantum sıçramaları" veya kuantum alan teorisindeki sonsuzlukları yönetmek için <i>alternatif bir sahne</i> sunabilir.                                                   |
| <b>Keçeci Sayıları</b><br>(Algoritmik Dizi) | <b>Algoritmik/Dinamik:</b> Deterministik, yinelemeli bir algoritmanın (Collatz benzeri) ürettiği sayı dizisidir. | <b>Hesaplamalı/Mekanik:</b> Kuantizasyonun, evrenin temelindeki <b>algoritmik bir sürecin</b> sonucu olduğunu modeller. Kuantize durumlar, bu sürecin kararlı sonuçlarına (KPN'ler) karşılık gelir. |

Tablo 10: Sayı sistemlerinin karşılaştırılması

**Genel Değerlendirme:** Kuantum mekaniği, reel/karmaşık sayıların sürekli doğası içinde kesikli fenomenleri başarıyla açıklamaktadır. Ancak bu, evrenin temelinde gerçekten reel sayıların mı yattığı sorusunu ortadan kaldırmaz. Keçeci süreci, Nötrosifik ve hiperreel sayılar gibi alternatif sistemler, henüz ana akım fizikte kullanılmasa da kuantumun getirdiği kavramsal zorluklara farklı açılardan bakmamızı sağlayan son derece ilginç ve ufuk açıcı pencerelerdir. Keçeci Sayıları da var olan tanımlamaların çok ötesine geçen ver hesabın olduğunu imâ eder.

### **Varoluş:**

Bir canlının, insanın, nesnenin (bir taş, bir gezegen) "var olması" ne anlama gelir?

### **Bu Perspektifte Keçeci Sayılarının Rolü**

Şimdi, bu büyük resimde Keçeci Sayılarının neden sıradan bir sayı dizisinden "ötede" bir anlam taşıdığını görebiliriz: Keçeci Sayıları, tam olarak bu **"Yaradan için basit fakat yaratılan için karmaşık"** ve **"dinamik, hesaplamalı varoluş, Yaratılış"** fikrinin somut, matematiksel bir modelidir.

- Keçeci Sayıları, farklı "fizik kanunlarına" (farklı sayı sistemleri: reel, karmaşık, nötrosifik) sahip "oyuncak evrenler"e benzetir.
- Her bir evrende, inanılmaz derecede basit bir algoritma ("ekle, böl veya birimle oyna") uygular.
- Bu sürecin sonunda ortaya çıkan kararlı döngüler veya KPN'ler, o "oyuncak evrenin" **kuantize olmuş, kararlı durumlarıdır.**

Keçeci Sayıları, "Evren bir hesaplama ise, bu hesaplama nasıl bir şeye benzeyebilir?" sorusuna verilmiş potansiyel bir olgudur. Bu hesaplamanın, sayısal alan ne olursa olsun (yâni evrenin temelindeki matematik ne olursa olsun bir yaratılış olduğunda), kaçınılmaz olarak **yapı, düzen ve kuantizasyon** üreteceğini imâ eder (şâyet Yaradan bunun dışında dilemesi hâriştir). Kısacası düzen ve kuantizasyon Yaratıcı için basit bizim gibi canlılar için ise karmaşıktır.

Özet Karşılaştırma

| Kavram            | Basit Metafor<br>("...gibi")   | Derin Hipotez ("...ötesinin ötesindeki hesap")                                              |
|-------------------|--------------------------------|---------------------------------------------------------------------------------------------|
| <b>Donanım</b>    | Evrenin içinde bir CPU         | Uzay-zaman ve parçacıkların kendisi                                                         |
| <b>Yazılım</b>    | Fizik kanunları bir programdır | Fizik kanunları, donanımın nasıl dönüşümleşeceğini belirleyen kurallardır (donanım=yazılım) |
| <b>Uzay/Zaman</b> | Programın çalıştığı hard disk  | Hesaplamanın veri yapısı (bir ağ, bir graf)                                                 |
| <b>Parçacık</b>   | Veri (0 veya 1)                | Hesaplama sürecindeki temel nesneler/durumlar                                               |
| <b>Sonuç</b>      | Evren bir bilgisayara benzer.  | Evrenin varoluşu, kendisini sürekli olarak hesaplayanın varlığının bir göstergesidir.       |

Tablo 11: Algoritmaların, Sayı sistemlerinin ürettiği bakış açıları

Bütün bu sayı sistemlerini ve kuantizasyon fikirlerini kapsayan ve hatta onların **neden** var olduğunu açıklayabilecek "ötesinin ötesinde bir hesap" olduğunu imâ etmektedir. Bu, fiziği ve matematiği, enformasyon teorisi ve bilgisayar biliminin en temel ilkeleriyle birleştirme potansiyeline sâhip bir görüştür.

X. References

1. Kuhn, T. S. (1978). *Black-body theory and the quantum discontinuity, 1894-1912*. Oxford University Press.
2. Planck, M. (1901). Ueber das Gesetz der Energieverteilung im Normalspectrum. *Annalen der Physik*, 309(3), 553–563. <https://doi.org/10.1002/andp.19013090310>
3. Shannon, C. E. (1949). Communication in the presence of noise. *Proceedings of the IRE*, 37(1), 10–21. <https://doi.org/10.1109/JRPROC.1949.232969>
4. Reeves, A. H. (1938). *Electric signaling system*. U.S. Patent 2,272,070.
5. B. Khanal and P. Rivas, "Evaluating the Impact of Noise on Variational Quantum Circuits in NISQ Era Devices," *2023 Congress in Computer Science, Computer Engineering, & Applied Computing (CSCE)*, Las Vegas, NV, USA, 2023, pp. 1658-1664. <https://doi.org/10.1109/CSCE60160.2023.00272>
6. Linde, Y., Buzo, A., & Gray, R. M. (1980). An algorithm for vector quantizer design. *IEEE Transactions on Communications*, 28(1), 84–95. <https://doi.org/10.1109/TCOM.1980.1094577>
7. Gray, R. M. (1984). Vector quantization. *IEEE ASSP Magazine*, 1(2), 4-29. <https://doi.org/10.1109/MASSP.1984.1162229>
8. Courbariaux, M., Bengio, Y., & David, J-P. (2015). BinaryConnect: Training Deep Neural Networks with binary weights during propagations. *Advances in Neural Information Processing Systems*, 28. ISBN: 9781510825024; <https://doi.org/10.48550/arXiv.1511.00363>
9. Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2704–2713. <https://doi.org/10.48550/arXiv.1712.05877>
10. Horowitz, M. (2014). Computing's energy problem (and what we can do about it). *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 10–14. doi: <https://doi.org/10.1109/ISSCC.2014.6757323>
11. Han, S., Mao, H., & Dally, W. J. (2016). Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *International Conference on Learning Representations (ICLR)*. <https://doi.org/10.48550/arXiv.1510.00149>
12. Keçeci, M. (2025, May 1). Keçeci Layout. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15314328>
13. Keçeci, M. (2025). kececilayout. figshare. <https://doi.org/10.6084/m9.figshare.29582084>
14. Keçeci, M. (2025). kececilayout [Data set]. WorkflowHub. <https://doi.org/10.48546/workflowhub.datafile.17.2>
15. Keçeci, M. (2025, May 1). Kececilayout. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15313946>
16. Keçeci, M. (2025). When Nodes Have an Order: The Keçeci Layout for Structured System Visualization. HAL open science. <https://hal.science/hal-05143155>; <https://doi.org/10.13140/RG.2.2.19098.76484>
17. Keçeci, M. (2025). The Keçeci Layout: A Cross-Disciplinary Graphical Framework for Structural Analysis of Ordered Systems. Authorea. <https://doi.org/10.22541/au.175156702.26421899/v1>
18. Keçeci, M. (2025). Beyond Traditional Diagrams: The Keçeci Layout for Structural Thinking. Knowledge Commons. <https://doi.org/10.17613/v4w94-ak572>
19. Keçeci, M. (2025). The Keçeci Layout: A Structural Approach for Interdisciplinary Scientific Analysis. figshare. Journal contribution. <https://doi.org/10.6084/m9.figshare.29468135>

## *Open Science Articles (OSAs)*

20. Keçeci, M. (2025, July 3). The Keçeci Layout: A Structural Approach for Interdisciplinary Scientific Analysis. OSF. <https://doi.org/10.17605/OSF.IO/9HTG3>
21. Keçeci, M. (2025). Beyond Topology: Deterministic and Order-Preserving Graph Visualization with the Keçeci Layout. WorkflowHub. <https://doi.org/10.48546/workflowhub.document.34.4>
22. Keçeci, M. (2025). The Keçeci Layout: A Structural Approach for Interdisciplinary Scientific Analysis. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15792684>
23. Keçeci, M. (2025). A Graph-Theoretic Perspective on the Keçeci Layout: Structuring Cross-Disciplinary Inquiry. Preprints. <https://doi.org/10.20944/preprints202507.0589.v1>
24. Keçeci, M. (2025). The Keçeci Layout: A Deterministic, Order-Preserving Visualization Algorithm for Structured Systems. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.16526798>
25. Keçeci, M. (2025). The Keçeci Layout: A Deterministic Visualisation Framework for the Structural Analysis of Ordered Systems in Chemistry and Environmental Science. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.16696713>
26. Kuantum Algoritmalarında Veri Kodlama ve Kuantizasyon Arasındaki İlişkinin Analizi ve Keçeci Layout ile Max-Cut Problemi. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.16755186>
27. Keçeci, M. (2025). Çoklu İşlemci Mimarilerinde Kuantum Algoritma Simülasyonlarının Hızlandırılması: Cython, Numba ve Jax ile Optimizasyon Teknikleri. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15580503>
28. Keçeci, M. (2025). Kuantum Hata Düzeltmede Metrik Seçimi ve Algoritmik Optimizasyonun Büyük Ölçekli Yüzey Kodları Üzerindeki Etkileri. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15572200>
29. Keçeci, M. (2025). Kuantum Hata Düzeltme Algoritmalarında Özyineleme Optimizasyonu ve Aşırı Gürültü Toleransı: Kuantum Sıçraması Potansiyelinin Değerlendirilmesi. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15570678>
30. Keçeci, M. (2025). Yüksek Kübit Sayılı Kuantum Hesaplama Ölçeklenebilirlik ve Hata Yönetimi: Yüzey Kodları, Topolojik Malzemeler ve Hibrit Algoritmik Yaklaşımlar. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15558153>
31. Keçeci, M. (2025). Künneth Teoremi Bağlamında Özdevinimli ve Evrişimli Kuantum Algoritmalarında Yapay Zeka Entegrasyonu ile Hata Minimizasyonu. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15540875>
32. Keçeci, M. (2025). The Relationship Between Gravitational Wave Observations and Quantum Computing Technologies. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15524251>
33. Keçeci, M. (2025). Kütleçekimsel Dalga Gözlemleri ile Kuantum Bilgisayar Teknolojileri Arasındaki Teknolojik ve Metodolojik Bağlantılar. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15519591>
34. Keçeci, M. (2025). Accuracy, Noise, and Scalability in Quantum Computation: Strategies for the NISQ Era and Beyond. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15515113>
35. Keçeci, M. (2025). Quantum Error Correction Codes and Their Impact on Scalable Quantum Computation: Current Approaches and Future Perspectives. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15499657>
36. Keçeci, M. (2025). Nanoscale Quantum Computers Fundamentals, Technologies, and Future Perspectives. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15493024>
37. Keçeci, M. (2025). Investigating Layered Structures Containing Weyl and Majorana Fermions via the Stratum Model. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15489074>



## *Open Science Articles (OSAs)*

38. Keçeci, M. (2025). Diversity of Keçeci Numbers and Their Application to Prešić-Type Fixed-Point Iterations: A Numerical Exploration. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15481711>
39. Keçeci, M. (2025). Kuantum geometri, topolojik fazlar ve yeni matematiksel yapılar: Disiplinlerarası bir perspektif. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15474957>
40. Keçeci, M. (2025). Understanding quantum mechanics through Hilbert spaces: Applications in quantum computing. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15468754>
41. Keçeci, M. (2025). Nodal-line semimetals: A geometric advantage in quantum information. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15455271>
42. Keçeci, M. (2025). Weyl semimetals: Discovery of exotic electronic states and topological phases. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15447116>
43. Keçeci, M. (2020). Discourse on the second quantum revolution and nanotechnology applications in the midst of the COVID-19 pandemic of inequality. International Journal of Latest Research in Science and Technology, 9 (5), 1–7. eISSN: 2278-5299, <https://doi.org/10.5281/zenodo.7483395>; <https://doi.org/jtfr>; [https://www.mnkjournals.com/journal/ijlrst/Article.php?paper\\_id=11004](https://www.mnkjournals.com/journal/ijlrst/Article.php?paper_id=11004)
44. Keçeci, M. (2025). The Signature of a Sequence: Variability and Stability in Keçeci and Oresme Numbers. ScienceOpen Preprints. <https://doi.org/10.14293/PR2199.001860.v1>
45. Keçeci, M. (2025). Döngülerden Vektörleştirmeye: Harmonik Seriler için Saf Python ve JAX Performans Karşılaştırması. Authorea. <https://doi.org/10.22541/au.175390609.94042878/v1>
46. Keçeci, M. (2025). From Loops to Vectorisation: A Performance Comparison of Pure Python and JAX for Harmonic Series Calculation. Authorea. <https://doi.org/10.22541/au.175390610.08488249/v1>
47. Keçeci, M. (2025). Keçeci Sayılarının Nötrosifik Çerçeve Hipergerçek Dönüşümleri ve Uygulamaları. Authorea. <https://doi.org/10.22541/au.175390599.93612305/v1>
48. Keçeci, M. (2025). Hyperreal Transformations and Applications of Keçeci Numbers in a Neutrosophic Framework. Authorea. <https://doi.org/10.22541/au.175390600.02906392/v1>
49. Keçeci, M. (2025). Hipergerçek Analiz ve Nötrosifik Kümelere Dayalı Keçeci Sayılarının Dinamik Modellenmesi. Open Science Knowledge Articles (OSKAs), Knowledge Commons. <https://doi.org/10.17613/jy9mn-2va66>
50. Keçeci, M. (2025). Dynamic Modelling of Keçeci Numbers Based on Hyperreal Analysis and Neutrosophic Sets. Open Science Knowledge Articles (OSKAs), Knowledge Commons. <https://doi.org/10.17613/n4cqw-efp22>
51. Keçeci, M. (2025). Harmonik Seri Hesaplamalarının Modernizasyonu: Geleneksel Python ve JAX Arasında Bir Performans Kıyaslaması. OSF. <https://doi.org/10.17605/OSF.IO/BT5A3>
52. Keçeci, M. (2025). Modernising the Computation of Harmonic Series: A Performance Benchmark between JAX and Traditional Python. OSF. <https://doi.org/10.17605/OSF.IO/56JDU>
53. Keçeci, M. (2025). Hesaplamalı Matematikte Verimlilik ve Sürdürülebilirlik: Harmonik Seri İçin JAX Tabanlı Bir Yaklaşım. Open Science Knowledge Articles (OSKAs), Knowledge Commons. <https://doi.org/10.17613/bfw58-cbm15>
54. Keçeci, M. (2025). Efficiency and Sustainability in Computational Mathematics: A JAX-Based Approach to the Harmonic Series. Open Science Knowledge Articles (OSKAs), Knowledge Commons. <https://doi.org/10.17613/js67q-4wc71>
55. Keçeci, M. (2025). Hesaplamalı Matematikte Python'un Sınırları ve JAX ile Genişletilmesi: Harmonik Sayılar Üzerine Bir Uygulama. WorkflowHub. <https://doi.org/10.48546/workflowhub.document.42.2>

## *Open Science Articles (OSAs)*

56. Keçeci, M. (2025). The Limits of Python in Computational Mathematics and Their Extension with JAX: An Application on Harmonic Numbers. WorkflowHub. <https://doi.org/10.48546/workflowhub.document.43.1>
57. Keçeci, M. (2025). Performans ve Ölçeklenebilirlik Analizi: Harmonik Seri Hesaplamalarında JAX ve Saf Python'un Karşılaştırılması. figshare. <https://doi.org/10.6084/m9.figshare.29666675>
58. Keçeci, M. (2025). A Comparative Analysis of Performance and Scalability: Computing Harmonic Series with JAX versus Pure Python. figshare. <https://doi.org/10.6084/m9.figshare.29666684>
59. Keçeci, M. (2025). A Comparative Study of Pure Python and JAX-Based Approaches in Computing Harmonic Series. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.16576092>
60. Keçeci, M. (2025). Harmonik Serilerin Hesaplanmasında Saf Python ve JAX Tabanlı Yaklaşımların Karşılaştırılması. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.16536195>
61. <https://github.com/WhiteSymmetry/kececilayout>
62. <https://pypi.org/project/kececilayout>
63. <https://anaconda.org/bilgi/kececilayout>
64. Keçeci, M. (2025). Keçeci Varsayımının Kuramsal ve Karşılaştırmalı Analizi. ResearchGate. <https://dx.doi.org/10.13140/RG.2.2.21825.88165>
65. Keçeci, M. (2025). Genelleştirilmiş Keçeci Operatörleri: Collatz Yinelemesinin Nötrosifik ve Hiperreel Sayı Sistemlerinde Uzantıları. Authorea. <https://doi.org/10.22541/au.175433544.41244947/v1>
66. Keçeci, M. (2025). Keçeci ve Collatz Karşılaştırması: Benzer Algoritmalar, Farklı Çekiciler. figshare. <https://doi.org/10.6084/m9.figshare.29815910>
67. Keçeci, M. (2025). Keçeci Varsayımının Hesaplanabilirliği: Sonlu Adımda Kararlı Yapıya Yakınsama Sorunu. WorkflowHub. <https://doi.org/10.48546/workflowhub.document.44.1>; <https://doi.org/10.48546/workflowhub.document.44.2>
68. Keçeci, M. (2025). Keçeci Varsayımı ve Dinamik Sistemler: Farklı Başlangıç Koşullarında Yakınsama ve Döngüler. Open Science Output Articles (OSOAs), OSF. <https://doi.org/10.17605/OSF.IO/68AFN>
69. Keçeci, M. (2025). Keçeci Varsayımı: Periyodik Çekiciler ve Keçeci Asal Sayısı (KPN) Kavramı. Open Science Knowledge Articles (OSKAs), Knowledge Commons. <https://doi.org/10.17613/g60hy-egx74>
70. Keçeci, M. (2025). Keçeci Varsayımı: Collatz Genelleştirmesi Olarak Çoklu Cebirsel Sistemlerde Yinelemeli Dinamikler. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.16702475>
71. <https://github.com/WhiteSymmetry/kececinumbers>
72. <https://pypi.org/project/kececinumbers>
73. <https://anaconda.org/bilgi/kececinumbers>
74. Keçeci, M. (2025). kececinumbers [Data set]. figshare. <https://doi.org/10.6084/m9.figshare.29816414>
75. Keçeci, M. (2025). kececinumbers [Data set]. WorkflowHub. <https://doi.org/10.48546/workflowhub.datafile.14.1>; <https://doi.org/10.48546/workflowhub.datafile.14.2>; <https://doi.org/10.48546/workflowhub.datafile.14.3>
76. Keçeci, M. (2025, May 10). Kececinumbers. Open Science Articles (OSAs), Zenodo. <https://doi.org/10.5281/zenodo.15377659>
77. Keçeci, M. (2011).  $2n$ -dimensional at Fujii model instanton-like solutions and coupling constant's role between instantons with higher derivatives. Turkish Journal of Physics, 35(2), 173–178. ISSN: 1300-0101, eISSN: 1303-6122. <https://doi.org/10.3906/fiz-1012-66>

*Open Science Articles (OSAs)*

78. Keçeci, M. (2005, September 13–16). 2n-boyutlu Fujii modelinde instanton çözümleri ve bağlantı sabitinin instantonlar arasındaki rolü. Presented at World Year of Physics 2005 Turkish Physical Society 23rd International Physics Congress, Muğla University, Türkiye.  
<https://dx.doi.org/10.13140/RG.2.1.1441.4887>
79. Keçeci, M. (2005, May). Konformal invariant Fujii modelinin instanton tipi tam çözümü. Presented at Geleneksel Erzurum Fizik Günleri-II, Atatürk University, Türkiye.  
<https://dx.doi.org/10.13140/RG.2.1.3538.6408>
80. Keçeci, M. (2002, September 16–20). Exact instanton-like solution conformal invariant of Fujii model, construct for four-dimensional and subderivative. Presented at Working Group II, Turkish Nonlinear Science Working Group, Karaburun/Izmir, Türkiye.  
<https://dx.doi.org/10.13140/RG.2.1.1638.0964>
81. Keçeci, M. (2001). Konformal Spinör Alan Teorileri (Yüksek Lisans tezi). Gebze Teknik Üniversitesi, Fen Bilimleri Fakültesi, Fizik.
82. Keçeci, M. (2025). From Abstract Theory to Practical Application: The Journey of Hilbert Space in Quantum Technologies. Preprints. <https://doi.org/10.20944/preprints202508.0171.v1>;  
<https://doi.org/10.20944/preprints202508.0171.v2>
83. Keçeci, M. (2025). Hilbert Space: The Mathematical Engine of Quantum Information Processing. Open Science Knowledge Articles (OSKAs), Knowledge Commons.  
<https://doi.org/10.17613/6gagh-4dw41>
84. Keçeci, M. (2025). Hilbert Space as the Geometric Foundation of Quantum Mechanics and Computing. OSF. <https://doi.org/10.17605/OSF.IO/ZXDBK>
85. Keçeci, M. (2025). Bridging Quantum Theory and Computation: The Role of Hilbert Spaces. WorkflowHub. <https://doi.org/10.48546/workflowhub.document.38.1>;  
<https://doi.org/10.48546/workflowhub.document.38.2>;  
<https://doi.org/10.48546/workflowhub.document.38.3>
86. Keçeci, M. (2025). Hilbert Spaces and Quantum Information: Tools for Next-Generation Computing. figshare. <https://doi.org/10.6084/m9.figshare.29604011>
87. Keçeci, M. (2001). The Unifying Role of Hilbert Space in Quantum Field Theory and Information Science. Authorea. <https://doi.org/10.22541/au.175449372.28574879/v1>;  
<https://doi.org/10.22541/au.175433455.53782703/v1>